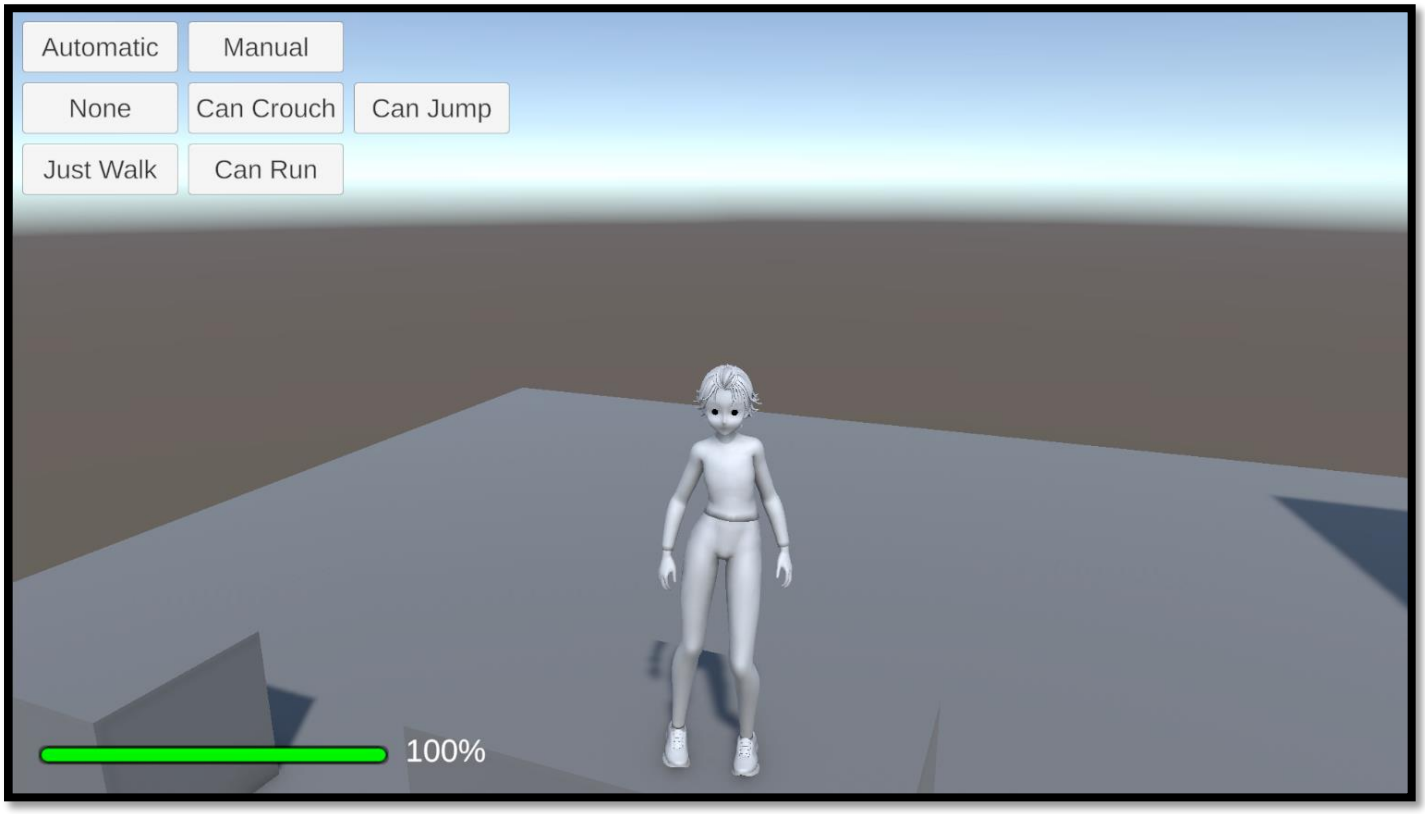

Player Controller

Documentation

_____

Creator: Lucas Gomes Cecchini

Online Name: AGAMENOM

Overview

This document will help you to use **Player Controller** for **Unity**.

With it, you can create a gameplay for your game with a modular system that's compatible with Unity's **Input System**.

It also comes with some **Predefined Controllers**.

Instructions

You can get more information from the Playlist on YouTube:

https://www.youtube.com/playlist?list=PL5hnfx09yM4LxhQjXoLlwwvmEuMW9LySa

Script Explanations

Read Only Attribute

This script is a small Unity editor extension that creates a **“read-only field attribute”** for the Inspector. In practice, it lets you show values in the Unity Inspector without allowing the user to edit them.

Overall Idea

When you add the custom marker to a variable, Unity will still display it in the Inspector, but the field becomes **disabled (greyed out)** so it cannot be changed.

This is done using two parts:

- A **marker attribute** (does nothing by itself, only tags a field).
- A **custom inspector drawer** (controls how the field is drawn in the editor).

Everything inside the editor-only block only runs inside Unity Editor, not in builds.

Namespace

PlayerController.Attributes

This is just a logical grouping. It helps organize scripts so they don’t clash with others in larger projects.

ReadOnlyAttribute (the marker)

Purpose

This is a simple attribute used to tag variables.

What it does:

- It does not store logic.
- It does not store data.
- It only acts as a **flag** that tells Unity:
  “This field should be handled differently in the Inspector.”

When used:

When you decorate a field with this attribute, Unity recognizes it later in the editor system.

ReadOnlyDrawer (the Inspector logic)

This is the part that actually changes how the Inspector behaves.

It only exists inside the editor because of the editor-only condition block.

CustomPropertyDrawer link

This tells Unity:

“Whenever you see a field with ReadOnlyAttribute, use this drawer to display it.”

So it automatically overrides the default Inspector drawing behavior.

Method: OnGUI

This is the core function that Unity calls every time it needs to draw the field in the Inspector.

It receives three main inputs:

---

position

This represents the rectangle area on the screen where the field should be drawn.

- Think of it as: “Where to draw the UI element.”

---

property

This is the actual variable being shown in the Inspector.

It contains:

- The value of the variable
- Its type (int, float, string, etc.)
- Metadata (name, serialization info)

This is what Unity uses to actually render and manage the field.

---

label

This is the text shown next to the field in the Inspector.

Example:

- “Health”
- “Speed”
- “Player Name”

It is just the visible name.

Inside OnGUI (step-by-step behavior)

1. Disabling the GUI

The script first turns off interaction:

- This prevents any input or modification.
- Everything drawn after this becomes non-editable.

So even though the field is still visible, it becomes greyed out.

---

2. Drawing the property field

Unity then draws the actual Inspector field using its built-in rendering system.

At this moment:

- The field is visible
- The value is shown
- But editing is blocked because GUI is disabled

---

3. Re-enabling the GUI

After drawing the field, the script restores normal UI behavior.

This is very important because:

- If not restored, everything else in the Inspector would also become uneditable.
- This ensures only this specific field is affected.

UNITY_EDITOR block

Everything inside this block:

- Exists only inside the Unity Editor
- Is completely removed in builds

- Prevents unnecessary code from running in the final game

So:

- Runtime builds → ignore this entirely
- Editor → uses it to enhance Inspector behavior

How to use it

When you apply this attribute to a variable:

- Unity detects it in the Inspector
- Uses the custom drawer
- Displays the field normally
- But disables editing

So it is mainly used for:

- Debug values
- Runtime stats (health, position, etc.)
- Read-only configuration previews
- Safely exposing internal data

Summary of behavior

- Attribute = marks a field as read-only
- Drawer = controls how it is displayed
- OnGUI = renders the field in Inspector
- GUI.enabled = false → disables editing
- GUI.enabled = true → restores normal UI
- Editor-only block → ensures no runtime cost

Tag Dropdown Attribute
This script creates a **custom Unity Inspector dropdown for tags**, allowing string fields to be selected from Unity's tag list instead of being typed manually. It also adds validation, multi-object support, and warning feedback when the value is invalid.

Overall Concept

When you attach the attribute to a string field, Unity replaces the normal text input with:

- A dropdown menu of all Unity tags
- A warning system if the value is not a valid tag
- Proper support for selecting multiple objects at once

It only works inside the Unity Editor (not in builds).

Namespace

PlayerController.Attributes

This groups everything related to custom Inspector tools for player controller systems.

TagDropdownAttribute (Marker Attribute)

Purpose

This is a **label only**, used to mark a string field.

What it does:

- It does NOT store logic
- It does NOT store data
- It only signals Unity:
  “This field should use a tag dropdown instead of a normal text field”

So it is just a trigger for the custom drawer.

TagDropdownDrawer (Inspector logic)

This is the system that replaces how the field is drawn in the Inspector.

It is automatically used whenever Unity detects the attribute.

Method: OnGUI (Main Drawing Function)

This method is responsible for **drawing and controlling the Inspector UI**.

It receives:

position

Defines where on the Inspector the field is drawn.

- Basically the UI rectangle area.

property

This is the serialized field being edited.

It contains:

- The current string value
- Multi-object editing state
- Metadata about the field

This is the actual data being manipulated.

---

label

The visible name of the field in the Inspector.

Example:

- “Enemy Tag”
- “Target Tag”

Step-by-step behavior inside OnGUI

---

1. Type validation

The script first checks if the field is a string.

If NOT:

- It shows a message: “Use [TagDropdown] with strings.”
- Then stops execution for this field.

👉 This prevents incorrect usage (like applying it to integers or floats).

---

2. Begin property block

This enables Unity features like:

- Prefab overrides
- Undo system
- Multi-object editing support

So changes behave properly in Unity’s serialization system.

3. Get Unity tags

It fetches all available tags from Unity's internal tag system.

This is the list shown in the dropdown.

4. Read current state

Two important values are collected:

hasMultipleDifferentValues

- True if multiple objects are selected
- And they have different tag values

currentString

- The actual string stored in the field

5. Missing tag handling

The script checks:

- Is the current string empty?
- Does it exist in Unity's tag list?

If not:

- It creates a special entry:
  "Tag Missing (value)"

This ensures invalid values are still visible instead of disappearing.

6. Build dropdown list

A new list is created:

- First item → "missing tag" placeholder (if needed)
- Then → all Unity tags

So the dropdown always includes valid options + error state.

7. Find current index

The script tries to locate the current string inside the list.

If not found:

- It defaults to index 0 (missing entry)

This ensures the dropdown always has a selected value.

---

8. Tooltip-only label rendering

A hidden label field is drawn to preserve tooltip functionality without interfering with layout.

---

9. Multi-object visual handling

If multiple objects are selected:

- Unity shows a mixed-value indicator (like “—” or striped field)

This tells the user that values differ between objects.

---

10. Convert to UI options

The string list is converted into UI-friendly entries.

Each tag becomes a selectable dropdown item.

---

11. Dropdown rendering

A popup menu is drawn using:

- label name
- current index
- tag options

User can select a new tag from the list.

---

12. Reset mixed-value state

After drawing, the system resets the multi-object visual mode so it doesn’t affect other fields.

13. Apply changes

If the user selects a valid option:

- And it is not the “missing” entry
- And it differs from previous selection

Then:

- The string value is updated with the new tag

14. Warning message

If:

- The selected value is invalid (missing tag)
- And not in multi-edit mode

Then a warning box is shown below the field:

“String value does not match any tag!”

This helps the user fix invalid data.

15. End property block

Closes the Unity property scope safely.

Method: GetPropertyHeight

This controls how much vertical space the field takes in the Inspector.

Input: property

The serialized field being drawn.

Input: label

Field name (not heavily used here).

Behavior

Step 1: Get tag list

It fetches all Unity tags again.

Step 2: Check validity

If the current string is NOT a valid tag:

- Extra space is added

This is needed because:

- Warning box takes additional vertical space

Step 3: Return height

Two possible outputs:

Normal case:

- One line height (standard field size)

Invalid tag case:

- Three times line height
- Enough space for dropdown + warning message

Usage

When you use this attribute on a string field:

What happens in Inspector:

- The field becomes a dropdown instead of text input
- It shows all Unity tags automatically
- If the value is invalid:
  - It shows “Tag Missing”
  - It displays a warning box
- If multiple objects are selected:
  - It supports mixed values properly

Summary

- Attribute = marks a string field for special behavior
- Drawer = replaces default Inspector rendering
- OnGUI = builds dropdown UI, handles selection, validation, warnings
- GetPropertyHeight = adjusts layout space dynamically
- Result = safe, user-friendly tag selection system instead of manual typing

Validate Reference Attribute

This script builds a **Unity Inspector validation system for object references**, designed to visually warn the user when a required reference is missing.

Instead of silently allowing null references, it:

- Highlights the field visually
- Shows a contextual help message
- Can treat missing references as either warnings or errors

Overall Idea

When you apply the attribute to a field, Unity:

- Checks if the field is an object reference
- If it is empty, it highlights the field
- Displays a message below it explaining what is missing
- Optionally changes the severity (warning vs error)

Attribute: ValidateReferenceAttribute

This is the **configuration layer** of the system.

Purpose

It defines how the validation behaves, but does not draw anything itself.

Variable: UseError

Type meaning

A boolean setting that controls how serious the warning is.

Behavior:

- If true:
  - Missing reference is treated as an **error**
  - The Inspector shows a stronger visual warning (red tone + error message)
- If false:

- Missing reference is treated as a **warning**
- The Inspector shows a softer highlight (yellow/orange tone)

So this variable defines the **severity level of the validation feedback**.

Constructor

Input: useError

This sets the value of the severity option when the attribute is used.

Default behavior:

- If nothing is specified → it assumes error mode

So you can use it in three ways:

- Default → error mode
- Explicit true → error mode
- Explicit false → warning mode

Drawer: ValidateReferenceDrawer

This is the **visual and logic system** that controls how the field appears in the Inspector.

It is only active in the Unity Editor.

Method: OnGUI (Main Drawing Logic)

This method controls:

- How the field looks
- Whether it is highlighted
- Whether a message appears

It receives three inputs:

---

position

Defines the screen area where the field is drawn.

---

property

The actual serialized field being inspected.

This contains:

- The object reference value
- Type information
- Serialization state

label

The field name shown in the Inspector.

Step-by-step behavior

1. Type validation

The script first checks if the field is an object reference.

If it is NOT:

- It draws the field normally
- Stops further logic

👉 This prevents misuse on unsupported types like integers or strings.

2. Store original color

The current UI background color is saved.

This is important because:

- The script will temporarily change colors
- Then it must restore the original state

Without this, the entire Inspector UI would be affected.

3. Check if reference is empty

The system checks:

- Is the object reference null?

This defines the main condition:

- Empty → warning/error state
- Not empty → normal display

4. Read attribute settings

The script retrieves the attribute instance applied to the field.

This gives access to:

- Whether it should behave as error or warning

So each field can have different behavior independently.

5. Apply background color highlight

If the reference is empty:

- The field background is tinted

Two possible visual states:

Error mode:

- Soft red color
- Indicates critical missing reference

Warning mode:

- Soft yellow/orange color
- Indicates non-critical missing reference

This makes missing references visible instantly.

6. Define field drawing area

A rectangle is created to define:

- Position
- Width
- Height (single line)

This ensures the field is drawn correctly in layout.

7. Draw the field

The object reference field is rendered.

At this point:

- It may be colored (if missing)
- It behaves like a normal Unity object picker

---

8. Restore original color

After drawing:

- UI background color is reset

This prevents color bleeding into other fields.

---

9. If reference is missing → show help message

If the field is empty, additional UI is shown below it.

This message explains:

- What type of object is expected
- That the reference is missing

---

10. Determine object type name

The system tries to detect the type of the field.

If available:

- Uses actual field type name

If not:

- Falls back to "Unknown"

This makes the message context-aware.

---

11. Build help message

A message is created like:

“Put an item of type ‘X’ here!”

This gives clear feedback about what is required.

12. Calculate message height

Because messages can wrap into multiple lines:

- The system dynamically calculates required space

This prevents overlapping UI elements.

13. Define help box position

A second rectangle is created:

- Positioned below the field
- Includes spacing from the main field
- Sized based on calculated height

14. Choose message type

Depending on configuration:

Error mode

- Strong visual warning

Warning mode

- Softer visual warning

This affects icon + color in the help box.

15. Draw help box

The message is displayed under the field.

So the user sees:

- The field itself
- A clear explanation of the problem

Method: GetPropertyHeight

This method tells Unity how much vertical space the field needs.

Without it, the help box could overlap other fields.

Input: property

The serialized field

Input: label

Field name (not heavily used here)

Step-by-step behavior

---

1. Check type

If not object reference:

- Use default single-line height

This avoids unnecessary spacing.

---

2. Check if reference is empty

If NOT empty:

- Only one line is needed
- No warning message

---

3. Determine type name

Same logic as OnGUI:

- Uses real field type if available
- Otherwise “Unknown”

This ensures consistency between UI and message.

---

4. Build help content

The same message text is recreated:

- Used for measuring layout height

---

5. Calculate help box height

Unity calculates how tall the message box will be depending on:

- Text length
- Inspector width

This ensures proper layout even with long messages.

---

6. Return total height

If empty reference:

- Field height
-
    - spacing
-
    - help box height

If filled:

- Only field height

Usage

When you apply this attribute to a field:

If reference is assigned:

- Field looks normal
- No message appears

If reference is missing:

- Field is highlighted (red or yellow)
- Warning or error message appears below
- Extra space is allocated automatically

Summary

- Attribute defines **behavior mode (error or warning)**
- Drawer controls **visual feedback in Inspector**
- OnGUI:
  - Validates type
  - Highlights missing references
  - Shows contextual help box
- GetPropertyHeight:
  - Ensures correct spacing for UI elements

Scene Dropdown Manager (Editor)

This script creates a **Unity Editor-friendly scene selector system using a dropdown UI**, allowing scenes to be loaded directly from a ScriptableObject instead of relying on Build Settings.

It works both in **Edit Mode and Play Mode**, but only inside the Unity Editor (everything is wrapped in editor-only conditions).

Overall Idea

The system:

- Reads a list of scenes from a ScriptableObject
- Displays them in a UI dropdown
- Lets the user select one
- Loads the selected scene when a button is clicked

Unlike Unity's default workflow, it does NOT rely on Build Settings.

Class: SceneDropdownManagerEditor

This is the main controller that connects:

- UI (dropdown + button)
- Scene data (ScriptableObject list)
- Scene loading system

It inherits from a general Unity component so it can be attached to a GameObject.

Serialized Fields (Inspector References)

These are the external inputs of the system.

sceneDropdown

Purpose

A reference to a TMP dropdown UI component.

Role in system:

- Displays the list of scenes
- Lets the user select one scene
- Controls which scene will be loaded

So it is the **main selection interface**.

loadSceneButton

Purpose

A Unity UI Button.

Role:

- Triggers the scene loading process
- Calls the function that reads the dropdown selection

So it acts as the **execution trigger**.

sceneList

Purpose

A ScriptableObject that stores scene references.

Role:

- Contains all scenes available in the dropdown
- Replaces Build Settings dependency
- Acts as the data source

So it is the **data provider for the system**.

Private Field

scenePaths (List of strings)

Purpose

Stores the actual file paths of scenes.

Why it exists:

The dropdown only shows names, but loading requires full paths.

So this list:

- Maps dropdown index → scene file path
- Keeps UI and file system linked

Each index in this list corresponds directly to a dropdown option.

Public Properties

These are optional access points to modify or retrieve references externally.

SceneDropdown

Role:

- Gets or sets the dropdown UI reference

Used when other scripts need to:

- Replace UI dynamically
- Access dropdown state

LoadSceneButton

Role:

- Gets or sets the button reference

Allows external scripts to:

- Swap buttons
- Reassign functionality

SceneList

Role:

- Gets or sets the ScriptableObject containing scenes

Useful for:

- Changing scene sets at runtime (Editor tools)
- Reconfiguring scene lists dynamically

Unity Callback: Start

This method runs automatically when the object becomes active.

Step 1: Validate SceneList

The system checks:

- Is the SceneList assigned?
- Does it contain scenes?
- Is the list not empty?

If any of these fail:

- A warning is printed
- The system stops initialization

👉 This prevents null errors or empty dropdowns.

Step 2: Setup dropdown

If validation passes:

- The dropdown is populated with scene data

This builds the UI options.

Step 3: Register button click

The button is linked to the loading function.

So when clicked:

- It triggers scene loading logic

Method: SetupDropdown

This method builds the dropdown UI dynamically.

Step 1: Clear old data

- Removes old dropdown entries
- Clears stored scene paths

This ensures no duplicated or outdated data.

Step 2: Prepare containers

- Creates a list for scene names (UI display)
- Uses internal list for file paths (logic)

So there are two parallel lists:

- Names → shown to user
- Paths → used for loading

Step 3: Get current scene name

The system retrieves:

- The currently open scene in the editor

This is used to auto-select the correct dropdown entry.

Step 4: Default selection tracking

A variable is used to track:

- Which dropdown entry should be selected by default

Initially set to zero.

Step 5: Loop through scene list

For each scene stored in the ScriptableObject:

If scene is null:

- Skip it

Get scene path

The system fetches:

- Full file path from Unity Asset Database

This links the asset reference to a real file location.

Extract scene name

From the file path:

- Only the file name is kept
- Extension is removed

This becomes the visible dropdown label.

Store data

For each scene:

- Name → added to dropdown options
- Path → stored in internal list

This creates a synchronized index system.

Check active scene match

If the scene matches the currently open scene:

- That index becomes the default selection

This improves usability by highlighting the current scene.

Step 6: Apply dropdown data

After looping:

- All names are added to dropdown

- Default selection is applied
- UI is refreshed so it updates visually

Method: OnLoadSceneButtonClicked

This is the scene loading trigger.

Step 1: Validate selection

The system checks:

- Is the selected index valid?
- Is it within range of stored scene paths?

If invalid:

- A warning is shown
- Execution stops

Step 2: Get selected scene path

Using the dropdown index:

- The corresponding scene file path is retrieved

This connects UI selection → real scene file.

Step 3: Load scene depending on mode

The system behaves differently depending on Unity state:

If Unity is in Play Mode:

- Scene is loaded asynchronously
- Uses Play Mode-compatible loading system

This avoids breaking runtime execution.

If Unity is in Edit Mode:

- Scene is opened directly in the editor

- Replaces current active scene

This allows fast development switching.

Usage

To use this system:

1. Create a ScriptableObject containing scenes
2. Assign it to sceneList
3. Assign a TMP dropdown to sceneDropdown
4. Assign a UI button to loadSceneButton
5. Press Play or use in Editor

Then:

- Dropdown shows all scenes from ScriptableObject
- Selecting one updates internal index
- Clicking button loads the scene

Summary

- sceneDropdown → UI selector
- loadSceneButton → triggers loading
- sceneList → stores available scenes
- scenePaths → maps dropdown index to file paths

Core flow:

1. Load scene list
2. Build dropdown UI
3. Track selection
4. Load selected scene on button click

Scene List (Editor)

This script is a **simple editor-only data container** used to store Unity scenes in a structured way so they can be displayed in tools like dropdowns and scene managers.

It does not contain gameplay logic. Its only role is to **hold references to scenes inside the Unity Editor**.

Overall Idea

The script creates a **ScriptableObject asset** that:

- Stores a list of scenes (as assets, not strings or paths)
- Can be edited in the Inspector
- Can be used by other editor tools (like dropdown scene loaders)

It replaces the need for Unity Build Settings in custom workflows.

Class: SceneListEditor

This is the main container class.

It inherits from ScriptableObject, meaning:

- It exists as an asset file in the project
- It does not need to be attached to a GameObject
- It is designed for storing data, not behavior

CreateAssetMenu (Usage entry point)

This part defines how the asset is created inside Unity.

fileName

- Default name when creating the asset
- Example: “SceneList”

menuName

- Where it appears in Unity’s “Create” menu
- Example path:
  Tools → Player Controller → Developer → Scene List (Editor)

order

- Controls position in the creation menu
- Lower values appear higher in the list

Usage:

Right-click in Project window → Create → Scene List (Editor)

This generates a new asset of this type.

Scene Storage Field

---

scenes (List of Scene Assets)

Purpose

This is the core data of the system.

It stores:

- References to Unity scene files (not paths or names)
- Direct asset links to scenes in the project

Type meaning (conceptually)

Each entry in the list represents:

- A scene file inside the project
- Stored as an asset reference
- Recognized by Unity Editor systems

Tooltip

This gives a hint in the Inspector:

“List of scene assets to display in the dropdown.”

So when someone edits this asset:

- They understand it is meant for UI tools (like dropdowns)

UNITY_EDITOR block

Everything inside this block only exists in the editor.

Meaning:

- Included in Unity Editor only
- Removed completely in builds
- Prevents runtime usage of editor-only types

This is important because scene asset references:

- Do not exist in builds
- Only make sense during development

How it works in the system

This ScriptableObject is used by other scripts (like scene dropdown managers).

Flow of usage:

1. Developer creates SceneList asset
2. Developer drags scenes into the list
3. Another system reads this list
4. UI (dropdown) displays scene names
5. Scene loader uses those references to open scenes

So it acts as a **central registry of scenes for editor tools**.

Key Variables Summary

---

scenes

- A list of scene references
- Editable in Inspector
- Stores all available scenes for tooling systems
- Used as a source for dropdowns or scene loaders

Why this exists

Normally Unity uses Build Settings for scene management.

This system replaces that with:

- Custom scene collections
- More flexible grouping
- Editor-friendly workflows
- Easier tooling integration

Usage in practice

To use this system:

1. Create the asset from the menu
2. Add scenes into the list via Inspector
3. Use another script (like a dropdown manager) to read it
4. Select and load scenes dynamically in the Editor

Summary

- This script is a **data container only**
- It stores a **list of scene references**
- It is **editor-only**
- It is used by other tools to build:
  - Dropdowns
  - Scene switchers
  - Editor utilities

Player Prefab Manager

This script is an **Unity Editor utility system that adds custom menu actions to create and configure player prefabs automatically inside the Hierarchy**.

Instead of manually dragging prefabs into the scene, it:

- Finds the correct prefab in the project
- Instantiates it under the selected object
- Unpacks it for full editing
- Selects it automatically
- Puts it into rename mode (like pressing F2)

Overall Idea

The script acts like a **smart prefab spawning tool inside Unity Editor menus**.

It gives you options like:

- Base Player
- First Person Player
- Third Person Player
- Camera Controller

Each one:

- Searches for a specific prefab by name
- Instantiates it in the scene
- Automatically prepares it for editing

Class: PlayerPrefabManager

This is a **static utility class**, meaning:

- It cannot be attached to GameObjects
- It only contains tools and helper functions
- It is used purely in the Unity Editor

CORE METHOD: InstantiateAndSetupPrefab

This is the main system that handles everything.

It receives:

prefabName

Purpose

The name of the prefab to search in the project.

Role:

- Used as a search key inside Unity's asset database
- Determines which prefab will be instantiated

parentObject

Purpose

Defines where in the scene hierarchy the prefab will be placed.

Behavior:

- If a parent is provided → prefab becomes a child of it
- If null → prefab is placed at root level

This allows:

- Organized hierarchy placement
- Context-based spawning (inside selected object)

Step-by-step behavior

1. Search for prefab assets

The system searches the entire project for assets matching:

- The prefab name
- The type "Prefab"

This returns a list of possible matches.

2. Validate search results

If nothing is found:

- An error is printed
- Execution stops

This prevents null instantiation attempts.

3. Filter by folder location

Even if multiple matches exist, the script only accepts prefabs inside:

“Player Controller/Controllers”

This ensures:

- Only correct system prefabs are used
- Avoids accidental duplicates from other folders

4. Store valid prefab path

The first matching valid prefab path is stored.

This becomes the final chosen prefab.

5. Handle missing valid prefab

If no valid prefab is found in the required folder:

- Error is logged
- Process stops

This enforces strict project structure.

6. Warn about duplicates

If multiple prefabs with the same name exist:

- A warning is shown

- But the system still continues using the filtered one

This helps detect naming conflicts.

7. Load prefab asset

The prefab is loaded from its file path into memory.

If loading fails:

- Error is shown
- Execution stops

This ensures only valid assets are instantiated.

8. Determine parent transform

If a parent object exists:

- Use its transform

Otherwise:

- No parent is assigned

This defines hierarchy placement.

9. Instantiate prefab in scene

The prefab is created inside the scene using Unity’s prefab system.

At this point:

- The object appears in the Hierarchy
- It is still linked to prefab structure

10. Final setup call

After creation:

- The object is passed to a second method for cleanup and configuration

METHOD: CompletePrefabSetup

This method finalizes the object after creation.

---

prefabName

Purpose

Used for undo system naming.

Helps Unity display:

“Create Player Test”

in undo history.

---

instantiatedObject

Purpose

The newly created prefab instance in the scene.

This is the object being configured.

Step-by-step behavior

---

1. Null check

If object creation failed:

- Method stops immediately

Prevents crashes or invalid operations.

---

2. Register undo action

The object is registered in Unity’s undo system.

This allows:

- Ctrl+Z to remove the prefab instantly
- Editor-safe modifications

3. Unpack prefab completely

The prefab instance is fully detached from its original prefab link.

This means:

- It becomes fully editable
- No overrides or prefab restrictions remain

It is now a normal scene object.

4. Select object

The newly created object becomes the active selection.

This means:

- It is highlighted in Hierarchy
- Inspector automatically shows it

5. Delay rename activation

A small delay is added before triggering rename mode.

Why delay?

- Unity needs a frame to update selection first

Then:

- If the object is still selected
- The system simulates pressing F2

This automatically enters rename mode.

MENU SYSTEM (Menu Items)

These are **Unity Editor menu entries**.

They appear under:

GameObject → Tools → Player Controller → 3D → ...

Each menu method

All menu methods follow the same pattern:

They call:

- InstantiateAndSetupPrefab
- Pass a specific prefab name
- Use currently selected object as parent

CreateBasePlayer

Creates:

- Base 3D player prefab

CreateBasePlayerSideView

Creates:

- Side view player prefab

CreateBasePlayerFirstPerson

Creates:

- First person player prefab

CreateBasePlayerThirdPerson

Creates:

- Third person player controller prefab

CreateBasePlayerThirdPersonCameraController

Creates:

- Third person camera controller prefab

Usage

To use this system:

1. Open Unity Editor
2. Select a GameObject in Hierarchy (optional parent)
3. Go to:
   GameObject → Tools → Player Controller
4. Choose a player type
5. Prefab is automatically:
    - Found
    - Instantiated
    - Unpacked
    - Selected
    - Ready for renaming

Summary

- System searches prefabs by name
- Filters them by folder structure
- Instantiates them under selected object
- Fully unpacks them for editing
- Selects and auto-renames them

Core workflow:

1. Menu item clicked
2. Prefab search
3. Path validation
4. Instantiation
5. Setup (undo, unpack, select, rename)

Camera State Manager

This script is a **camera state controller for Unity that dynamically enables or disables camera control based on input and/or scene conditions**. It is designed for systems like menus, UI overlays, or gameplay states where the camera should be temporarily locked or released.

Overall Idea

The script manages whether the camera is active by using three possible control methods:

- Manual control (called from other scripts)
- Input-based toggle (button press)

- Automatic monitoring of GameObjects (UI or gameplay triggers)

It also handles:

- Cursor locking/unlocking
- State transition smoothing (prevents flickering)
- Events for external systems to react to camera changes

Enum: UpdateType

This defines **how the system decides when to update the camera state**.

ManualToggle

- Camera is controlled externally
- No automatic updates
- Another script must call the state change

👉 Used for full control systems or cutscenes.

InputEvent

- Camera toggled via input action
- Example: button press

👉 Used for player-controlled switching (like opening menus).

Update

- Camera state is checked every frame

👉 Used when real-time continuous checking is needed.

FixedUpdate

- Camera state is checked at fixed intervals

👉 Used when physics timing consistency is preferred.

Serialized Fields (Inspector Inputs)

These define how the system behaves externally.

---

toggleAction (Input Action Reference)

Purpose

Defines the input used to toggle the camera.

Role:

- Listens for a specific button/action
- Triggers camera enable/disable when pressed

👉 Example: Escape key to open menu

---

updateType

Purpose

Controls which update mode is active.

Role:

- Determines when the system checks or changes camera state
- Directly uses the UpdateType enum

---

activationObjects (List of GameObjects)

Purpose

Defines objects that influence camera state.

Role:

- If ANY object in this list is active → camera turns off
- If ALL are inactive → camera turns on

👉 Common usage:

- UI menus
- Pause screens
- Interaction zones

---

debug (bool)

Purpose

Enables logging for debugging purposes.

Role:

- Prints state changes in console
- Helps track behavior during development

OnEnable (event)

Purpose

Triggered when camera becomes active.

Role:

- External systems can subscribe
- Example: re-enable player input

OnDisable (event)

Purpose

Triggered when camera becomes inactive.

Role:

- Example: disable player movement or UI changes

Private Fields (Internal State)

These are used to track system behavior over time.

---

isCameraActive

Stores current camera state:

- true → camera enabled
- false → camera disabled

👉 Prevents unnecessary repeated state changes.

lastActivationState

Tracks previous frame’s activation condition from objects.

Used to detect:

- Changes in UI or game state

lastUpdateTime

Stores the last time the system checked state.

Used to:

- Measure time differences
- Support smooth transitions

activeDuration

Tracks how long activation objects have stayed active.

Used for:

- Preventing instant toggles
- Adding delay before disabling camera

inactiveDuration

Tracks how long activation objects have stayed inactive.

Used for:

- Delayed reactivation of camera

toggleEvent

Stores the input system subscription.

Used to:

- Listen for input events
- Clean up properly when object is destroyed

Properties

These allow external scripts to access or modify internal settings safely.

ToggleAction

Gets or sets the input used for toggling camera state.

ModeType

Gets or sets the update behavior mode.

ActivationObjects

Gets or sets the list of controlling objects.

DebugLog

Gets or sets debug logging state.

Unity Lifecycle Methods

Awake

Runs before Start.

Behavior:

- Checks if input action exists
- If missing:
  - Logs warning
  - Disables script completely

👉 Prevents runtime errors from missing configuration.

Start

Initial setup.

Behavior:

- Sets camera active by default
- Registers input event listener

If input mode is enabled:

- Pressing the assigned button toggles camera state

OnDestroy

Cleanup method.

Behavior:

- Disposes input event subscription

👉 Prevents memory leaks or ghost input listeners

Update

Runs every frame.

Behavior:

- Only active if mode is Update
- Calls object-based evaluation method

FixedUpdate

Runs at fixed intervals.

Behavior:

- Only active if mode is FixedUpdate
- Calls same evaluation system

Core Method: SetCameraState

This is the **main function that changes camera state**.

---

active (input parameter)

Defines desired state:

- true → enable camera
- false → disable camera

Behavior step-by-step

1. Prevent redundant changes

If state is already correct:

- Do nothing

👉 Improves performance

---

2. Update internal state

Stores new camera state.

---

3. Cursor control

If camera is active:

- Cursor is locked
- Cursor is hidden

If inactive:

- Cursor is unlocked
- Cursor becomes visible

👉 This is typical for FPS/menu switching systems

---

4. Trigger events

- If enabled → OnEnable event fires
- If disabled → OnDisable event fires

👉 Allows other systems to react automatically

---

5. Debug logging

If enabled:

- Logs state change in console

Core Method: UpdateCameraStateFromObjects

This is the **automatic state evaluator**.

Step 1: Check if list is empty

If no objects exist:

- Exit immediately

Step 2: Detect activation state

Checks if ANY object in list is active.

Result:

- true → something is active
- false → all inactive

Step 3: Time tracking

Uses real time to measure:

- How long state has been stable

Step 4: Detect state change

If activation state changed:

- Reset timers
- Store new state
- Stop processing

👉 Prevents flickering and instability

Step 5: If objects are active

- Increase active timer
- After 0.1 seconds:
    - Disable camera

👉 Adds delay to prevent instant toggling

Step 6: If objects are inactive

- Increase inactive timer
- After 0.1 seconds:
    - Enable camera

👉 Smooth reactivation behavior

Core Method: IsAnyActivationObjectActive

Checks all objects in the list.

Returns:

- true → at least one object is active
- false → none are active

Uses:

- simple existence check per object
- ignores null references

Usage

This system is typically used for:

UI menus

- When menu opens → camera disables
- When menu closes → camera enables

Pause systems

- Pause screen activates camera lock

Interaction zones

- Enter UI mode → disable camera control

Input toggle systems

- Press key → toggle camera mode

Summary

- Enum controls update strategy
- InputAction handles toggle input
- GameObject list defines activation triggers
- Events notify external systems
- Cursor is automatically managed
- Timing system prevents flicker
- Supports manual, input, or automatic modes

Core behavior flow:

1. Detect input or object state
2. Evaluate activation conditions
3. Apply delay filtering
4. Enable or disable camera
5. Update cursor + trigger events

Footstep Sound Controller

This script is a **dynamic footstep and landing audio system for a player controller**, designed to react to movement state, grounding status, and movement speed. It produces more natural sound behavior by varying timing and pitch, and it can work with any movement controller through a flexible state adapter system.

Overall Idea

The system is responsible for:

- Playing footstep sounds while the player moves
- Adjusting timing depending on movement type (walk, run, crouch)
- Playing landing sounds when hitting the ground
- Randomizing audio pitch for realism
- Reading movement data from any controller using an abstraction layer

It does NOT directly depend on a specific movement script. Instead, it relies on an interface-based adapter system.

ENUM: MovementState

This defines all possible movement conditions used for sound selection.

None

- Player is not moving
- No footstep sounds are played

Walking

- Normal movement speed
- Standard footstep interval

Running

- Faster movement
- Shorter interval between steps

CrouchWalking

- Slow, quiet movement while crouched

CrouchRunning

- Fast crouched movement (rare but supported)

SERIALIZED FIELDS (Inspector Inputs)

These define how the sound system behaves.

playerController

Purpose

Reference to the movement script controlling the player.

Role:

- Source of movement data
- Must expose movement properties through an interface system

- Used indirectly via reflection adapter

👉 This allows compatibility with different controllers.

---

audioSource

Purpose

Unity component responsible for playing audio.

Role:

- Plays footstep and landing sounds
- Controls volume and pitch
- Central sound output system

---

audioMixerGroup

Purpose

Routes audio through Unity Audio Mixer.

Role:

- Controls global audio effects
- Allows grouping footsteps into audio categories

---

footstepVolume

Purpose

Base volume for all footstep sounds.

Role:

- Controls overall loudness
- Scales all playback equally

---

minimumPitch / maximumPitch

Purpose

Defines random pitch variation range.

Role:

- Prevents repetitive sound fatigue
- Makes footsteps feel natural

---

walkInterval

Purpose

Time between steps when walking normally.

Role:

- Controls pacing of walking audio

---

runInterval

Purpose

Time between steps when running.

Role:

- Faster rhythm for running movement

---

walkClips

Purpose

List of audio clips used for walking/running footsteps.

Role:

- Randomly selected per step
- Creates variation in sound

---

crouchWalkInterval / crouchRunInterval

Purpose

Timing values for crouched movement.

Role:

- Slower or different rhythm when crouching

crouchClips

Purpose

Audio clips used specifically for crouching footsteps.

Role:

- Separate sound set for stealth or low movement

playLandingSound

Purpose

Toggle for landing sound system.

Role:

- Enables or disables landing audio

landingClips

Purpose

Audio clips used when landing after a jump or fall.

Role:

- Random selection when landing occurs

PRIVATE FIELDS (Internal State)

playerState

Purpose

Interface-based abstraction of movement data.

Role:

- Reads movement info like:

- IsMoving
- IsRunning
- IsCrouching
- IsGrounded

👉 This allows compatibility with different controllers using reflection.

---

currentInterval

Purpose

Stores current footstep delay.

Role:

- Dynamically updated based on movement state

---

lastFootstepTime

Purpose

Stores last time a footstep was played.

Role:

- Prevents constant or overlapping sounds

---

wasGroundedLastFrame

Purpose

Tracks grounding state from previous frame.

Role:

- Detects transitions (air → ground or ground → air)

UNITY LIFECYCLE

Awake

Runs when object initializes.

Step-by-step behavior:

1. Validation

Checks:

- Player controller exists
- Audio source exists

If missing:

- Logs error

---

2. Create movement adapter

Creates a bridge system that:

- Reads movement data dynamically
- Uses reflection internally

👉 This allows compatibility with any controller.

---

3. Validate pitch range

Ensures:

- Minimum pitch is not higher than maximum

---

4. Configure audio source

Sets:

- No auto-play
- Volume level

---

5. Assign audio mixer (optional)

Routes sound through mixer group if assigned.

---

6. Initialize footstep timer

Ensures:

- First step can play immediately

FixedUpdate

Runs at fixed intervals (physics-safe timing).

Flow:

1. Check if player state exists
2. Handle landing logic
3. Determine movement state
4. Update footstep system

CORE SYSTEM: HandleLandingSound

This detects landing events.

Step 1: Check grounded state

Compares:

- Current grounded state
- Previous frame grounded state

Step 2: Detect change

If player:

- Just landed
- Or just left ground

Then trigger logic

Step 3: Play landing sound

If enabled:

- Random clip is selected
- Played with pitch variation

Step 4: Fallback behavior

If landing sounds disabled:

- Play normal footstep instead

Step 5: Update tracking state

Stores current grounded state for next frame

CORE SYSTEM: UpdateFootsteps

Controls footstep timing.

Step 1: Update interval

Sets correct delay based on movement type

Step 2: Validate conditions

Stops execution if:

- Player is not grounded
- Player is not moving

Step 3: Check timing

Only plays sound if:

- Enough time has passed since last step

Step 4: Play footstep

Triggers sound playback and updates timer

CORE SYSTEM: PlayFootstep

Selects correct sound set.

Logic:

- If crouching → use crouch clips
- Otherwise → use normal walk clips

Step:

- Random clip selected
- Sent to playback function

CORE SYSTEM: PlayFootstepClipWithPitch

Handles actual audio playback.

Step 1: Validate clip

Ensures clip exists

Step 2: Random pitch

Applies variation:

- Prevents repetitive audio pattern

Step 3: Play sound

Uses AudioSource one-shot playback:

- Does not interrupt other sounds
- Allows overlapping audio

STATE SYSTEM: GetMovementState

Determines current movement mode.

Logic order:

1. Not moving

→ None

2. Moving + crouching

→ crouch state (walk or run)

3. Moving normally

→ walk or run state

STATE SYSTEM: SetCurrentInterval

Maps movement state → timing value.

Mapping:

- Walking → walk interval
- Running → run interval
- Crouch walking → crouch walk interval
- Crouch running → crouch run interval
- None → disabled (infinite delay)

USAGE

This system is typically used in:

First-person controllers

- Footsteps match movement speed

Third-person systems

- Sound adapts to animations

Stealth mechanics

- Crouch uses different audio set

Jumping systems

- Landing sound improves feedback

SUMMARY

- MovementState defines behavior categories
- playerState provides movement data dynamically
- AudioSource handles playback
- Timing system controls step rhythm
- Pitch randomization adds realism
- Landing detection uses state transitions
- Works with any controller via abstraction layer

Core loop:

1. Read movement state
2. Determine timing
3. Detect landing
4. Choose correct audio set
5. Apply pitch variation
6. Play sound

Look Input Handler

This script is a **camera look input handler built on Unity's Input System**, designed to capture mouse or analog stick movement and expose it as a clean directional value that other systems (like a camera controller) can use.

It also includes runtime control, allowing input to be enabled or disabled dynamically (useful for menus, cutscenes, or UI states).

Overall Idea

The script:

- Listens to a "look" input action (mouse or right stick)
- Stores the input as a 2D vector (horizontal + vertical movement)
- Continuously updates this value while input is active
- Resets it when input stops
- Allows enabling/disabling input processing at runtime
- Cleans up event bindings properly when destroyed

Class: LookInputHandler

This is the main component that:

- Connects Unity Input System to gameplay logic
- Stores camera look direction
- Acts as a bridge between input and camera rotation systems

SERIALIZED FIELDS (Inspector Inputs)

lookAction

Purpose

Defines the input source used for camera look.

Role:

- Captures mouse movement or controller right stick
- Provided through Unity Input System
- Drives all look behavior

👉 Example inputs:

- Mouse delta
- Right analog stick

LookDirection

Purpose

Stores the current input direction.

Role:

- X axis → horizontal look (left/right)
- Y axis → vertical look (up/down)

Special note:

This value is marked as read-only in the Inspector:

- It can be seen
- But not edited manually

👉 This ensures it is always controlled by input only

PRIVATE FIELDS

lookInputEvent

Purpose

Stores the active input event configuration.

Role:

- Manages input subscription lifecycle
- Handles hold and release events
- Ensures proper cleanup

👉 This is the connection between Input System and this script

isPlayable

Purpose

Controls whether input is active or ignored.

Role:

- true → input is processed normally
- false → input is ignored and reset

👉 Used for:

- Menus
- Cutscenes
- Pausing camera control

PROPERTIES

LookAction

Purpose

Provides external access to input action reference.

Role:

- Allows swapping input dynamically
- Used by other scripts or systems

UNITY LIFECYCLE METHODS

Awake

Runs when object initializes.

Behavior:

- Checks if input action exists
- If missing:
  - Logs warning

👉 Prevents silent failure where camera input doesn't work

Start

This is where input binding is created.

Step 1: Create input binding

The system:

- Connects to the input action
- Only allows execution if isPlayable is true

Step 2: OnHold event

When input is active:

- Receives a 2D vector
- Updates LookDirection continuously

👉 This represents real-time camera movement

Step 3: OnReleased event

When input stops:

- LookDirection is reset to zero

👉 Prevents unwanted camera drift

OnDestroy

Runs when object is destroyed.

Behavior:

- Disposes input event system

👉 Prevents:

- Memory leaks
- Ghost input callbacks
- Invalid event references

CORE METHOD: TogglePlayable

This method controls whether input is active.

Parameter: enabled

Defines input state:

- true → input enabled
- false → input disabled

Behavior step-by-step

---

1. Update internal state

Stores whether input is allowed.

---

2. If disabled → reset input

If input is turned off:

- LookDirection is immediately set to zero

👉 This ensures:

- Camera stops instantly
- No leftover movement remains

INPUT FLOW (How it works overall)

When input is active:

1. Player moves mouse or stick
2. Input System sends Vector2 value
3. OnHold event triggers
4. LookDirection is updated continuously

When input stops:

1. Input System sends release event
2. OnReleased triggers
3. LookDirection is reset to zero

When input is disabled:

1. isPlayable becomes false
2. Input events are ignored
3. LookDirection is cleared

USAGE

This script is typically used in:

Camera controllers

- Smooth rotation based on LookDirection

First-person systems

- Mouse controls camera pitch and yaw

Third-person systems

- Camera orbit behavior

UI states

- Input disabled when menus are open

Cutscenes

- Prevents player camera movement

SUMMARY

- lookAction defines input source
- LookDirection stores live input vector
- lookInputEvent manages event lifecycle
- isPlayable controls input enable/disable state

Core behavior loop:

1. Read input action
2. Update direction while held
3. Reset when released
4. Allow runtime enable/disable
5. Clean up safely on destruction

Player Animation Controller 3D

This script is an **animation state controller for a 3D side-view character**, designed to ensure that only one animation plays at a time based on the player's movement, grounded state, and crouch status.

It acts as a bridge between a movement system and Unity’s Animator, converting gameplay state into clean animation transitions.

Overall Idea

The script:

- Reads player movement data (moving, running, crouching, grounded)
- Converts that data into a single animation state
- Ensures only one animation is active at any time
- Updates Unity Animator booleans accordingly
- Avoids redundant animation switching for performance and stability

---

Core Concept: AnimationState (Enum)

This defines all possible animation modes the character can be in.

Only one is active at any time.

Stopped

- Player is idle on the ground
- Not moving
- Not crouching

Walking

- Normal movement speed on ground

Running

- Faster movement on ground

Jumping

- Player is in the air

Crouching

- Idle crouch state (not moving)

Crawling

- Slow crouch movement

CrawlingRunning

- Fast crouch movement

INSPECTOR FIELDS (User Configuration)

playerController

Purpose

Reference to the movement system.

Role:

- Provides movement data like:
  - IsMoving
  - IsRunning
  - IsCrouching
  - IsGrounded

👉 This is the “source of truth” for animation decisions.

animator

Purpose

Unity Animator component controlling animations.

Role:

- Receives boolean parameters
- Switches animation states visually

Animator Parameter Strings

These define the **names of Animator booleans** inside Unity’s Animator Controller.

Each string corresponds to one animation state.

stopped

- Idle animation parameter

walking

- Walk animation parameter

running

- Run animation parameter

jumping

- Jump animation parameter

crouching

- Idle crouch animation parameter

crawling

- Crouch walk animation parameter

crawlingRunning

- Fast crouch movement animation

👉 These strings allow flexible Animator setups without hardcoding parameter names.

---

currentState (Debug Only)

Purpose

Stores the currently active animation state.

Role:

- Helps track what animation is currently active
- Prevents unnecessary switching

PRIVATE FIELDS

---

stateHashes

Purpose

Stores optimized Animator parameter IDs.

Role:

- Converts string parameter names into fast integer hashes
- Improves performance (avoids string lookup every frame)

👉 This is a common Unity optimization technique.

---

playerState

Purpose

Interface-based access to movement data.

Role:

- Reads movement state dynamically
- Works with any controller via adapter

UNITY LIFECYCLE METHODS

Awake

This method prepares everything before gameplay starts.

Step 1: Validation

Checks:

- playerController exists
- animator exists

If missing:

- Logs warnings

Step 2: Create movement adapter

Builds a bridge system that:

- Reads movement data from any controller
- Uses reflection internally

Step 3: Cache Animator parameters

Each animation state:

- Is converted from string → hash
- Stored in dictionary

👉 This makes animation switching fast and efficient

Step 4: Set initial state

- Default state is “Stopped”
- All animator booleans are set:
  - Only Stopped = true
  - All others = false

UPDATE METHOD

Runs every frame.

Step 1: Safety check

Ensures:

- playerController exists
- animator exists

If missing → stops execution

Step 2: Determine new state

Calls logic function that evaluates player movement.

Step 3: Compare states

If state changed:

- Disable old animation boolean

- Enable new animation boolean
- Update current state tracker

👉 This prevents constant animation resetting every frame

CORE LOGIC: DetermineState

This is the decision engine for animations.

Rule 1: Idle state

If:

- Not moving
- On ground
- Not crouching

→ Stopped

Rule 2: Ground movement

If:

- On ground
- Moving
- Not crouching

→ Walking or Running depending on speed

Rule 3: Air state

If:

- Not grounded
- Not crouching

→ Jumping

Rule 4: Crouch idle

If:

- On ground

- Crouching
- Not moving

→ Crouching

Rule 5: Crouch movement

If:

- On ground
- Crouching
- Moving

→ Crawling or CrawlingRunning

Fallback

If nothing matches:

- Keep current state

HOW ANIMATION SWITCHING WORKS

Step-by-step flow:

1. Read movement state
2. Convert into AnimationState
3. Compare with current state
4. If different:
    - Turn OFF previous animation bool
    - Turn ON new animation bool
5. Store new state

WHY THIS SYSTEM IS GOOD

---

1. Only one animation active

Prevents conflicting animations like:

- Walking + Jumping at same time

2. Performance optimized

Uses:

- Cached hashes instead of strings

3. Flexible input system

Works with any controller via adapter

4. Clean state logic

All decisions centralized in one method

5. Prevents unnecessary updates

Only changes Animator when state actually changes

USAGE

This script is typically used in:

Side-scrolling games

- Character movement animations

Platformers

- Jump / run / crouch transitions

Action games

- Smooth animation switching based on movement

Custom controllers

- Works with any movement system via interface adapter

SUMMARY

- AnimationState defines all animation modes
- playerController provides movement data
- animator receives boolean parameters
- stateHashes optimize performance
- Update only triggers changes when needed
- DetermineState decides what animation should play

Core loop:

1. Read player movement
2. Decide animation state
3. Compare with previous state
4. Switch Animator booleans if needed
5. Keep system stable and synchronized

Stamina Monitor

This script is a **stamina UI monitoring system** for a player controller. It takes stamina data from any compatible player script (even if the script is different), then displays it in a UI slider and optional text, while also changing colors based on stamina levels.

It is designed to be **data-driven, flexible, and controller-agnostic** using a reflection-based adapter system.

Overall Idea

The script:

- Reads stamina percentage from a player controller
- Updates a UI slider in real time
- Changes slider color depending on stamina level
- Optionally shows stamina as text
- Works with any controller exposing a stamina property
- Uses an adapter system to avoid hard dependencies

CORE COMPONENT: StaminaMonitor

This is the main class responsible for:

- Connecting player stamina data → UI display
- Updating visuals every frame
- Managing thresholds and colors

SERIALIZED FIELDS (Inspector Inputs)

---

playerController

Purpose

Reference to any player script that contains stamina data.

Role:

- Source of stamina value
- Must expose a stamina percentage property

👉 This makes the system reusable across different controllers.

---

maxStamina

Purpose

Defines the maximum stamina value used for normalization.

Role:

- Converts raw stamina into percentage form
- Used for UI scaling

---

minStaminaForRun

Purpose

Defines the minimum stamina required for running.

Role:

- Used to calculate "low stamina" threshold
- Controls when UI changes color

---

staminaSlider

Purpose

UI slider showing stamina visually.

Role:

- Displays stamina as a bar from 0 to 100%
- Main visual representation of stamina

---

emptyStaminaColor

Purpose

Color used when stamina is fully depleted.

Role:

- Signals exhaustion state (red by default)

---

lowStaminaColor

Purpose

Color used when stamina is low.

Role:

- Warns player that stamina is nearly depleted

---

normalStaminaColor

Purpose

Color used when stamina is healthy.

Role:

- Default UI state (green by default)

---

staminaText

Purpose

Optional text display for stamina percentage.

Role:

- Shows exact numeric value (e.g., 75%)

PRIVATE FIELDS

sliderFillImage

Purpose

Reference to the colored fill part of the slider.

Role:

- Allows changing only the fill color (not full UI)

minimumStaminaColorThreshold

Purpose

Converted threshold in percentage form.

Role:

- Used to decide when stamina is “low”

Formula:

- (min stamina for run ÷ max stamina) × 100

playerStamina

Purpose

Interface-based access to stamina data.

Role:

- Reads stamina value through abstraction
- Uses reflection internally via adapter

PROPERTIES

These allow external scripts to safely modify values.

Each one simply provides:

- Access (get)
- Optional modification (set)

They mirror the inspector fields.

UNITY LIFECYCLE METHODS

Start

This method initializes everything once the game begins.

Step 1: Calculate threshold

Converts stamina requirement into percentage:

- Example:
  If max stamina = 50
  and run threshold = 12.5
  → 25% threshold

👉 This defines when stamina becomes “low”

Step 2: Setup slider

If slider exists:

- Minimum value set to 0
- Maximum value set to 100
- Fill image reference is cached

👉 This prepares the UI for percentage display

Step 3: Setup stamina source

If player controller exists:

- Creates adapter system

If missing:

- Logs error

👉 This connects UI to player data safely

UPDATE METHOD (runs every frame)

This is the main UI update loop.

Step 1: Safety check

If no stamina source exists:

- Stop execution

Step 2: Read stamina value

Gets current stamina percentage:

- Value range: 0 to 100

Step 3: Update slider

If slider exists:

- Set slider value to stamina

Step 4: Change slider color

Color logic:

Case 1: stamina = 0

→ emptyStaminaColor

Case 2: stamina low threshold or below

→ lowStaminaColor

But only if not already empty

Case 3: stamina above threshold

→ normalStaminaColor

👉 This creates visual feedback for player condition

Step 5: Update text (optional)

If text exists:

- Displays stamina percentage

Formatting rules:

- Whole number → “75%”
- Decimal → “75.5%”

INTERFACES & ADAPTER SYSTEM

IPlayerStaminaInfo

Purpose

Defines required stamina data contract.

Role:

- Any controller must expose:
  - StaminaPercent

👉 This standardizes access across systems

PlayerStaminaAdapter

This is the bridge between UI and player controller.

Purpose

Reads stamina from any script dynamically.

How it works:

- Receives a MonoBehaviour reference
- Uses reflection-like helper
- Attempts to read:
    - StaminaPercent property

If successful:

- Returns stamina value

If not:

- Returns 0

👉 This makes the system:

- Flexible
- Decoupled
- Reusable

HOW THE SYSTEM WORKS (FLOW)

Initialization phase

1. Read inspector values
2. Calculate thresholds
3. Setup slider
4. Connect to player stamina source

Runtime phase (every frame)

1. Get stamina value

2. Update slider
3. Adjust color based on thresholds
4. Update text display

USAGE

This script is typically used in:

RPG games

- Stamina-based sprint systems

Survival games

- Fatigue mechanics

Action games

- Sprint / dodge resource system

UI HUD systems

- Real-time stamina visualization

SUMMARY

- playerController provides stamina data
- PlayerStaminaAdapter reads it dynamically
- staminaSlider displays visual bar
- staminaText shows numeric value
- colors change based on thresholds
- system updates every frame

Core loop:

1. Read stamina value
2. Convert to UI percentage
3. Update slider
4. Change color based on thresholds

5. Update text (if enabled)

Physics Update Broadcaster

This script is a **global physics and lifecycle event broadcaster combined with a gizmo rendering manager**, designed to centralize Unity update loops and also visualize physics sensors in the editor.

It acts as a **singleton system**, meaning only one instance exists across the entire game, and it provides shared events that other scripts can subscribe to instead of using their own update methods.

Overall Idea

The script has two main responsibilities:

1. Event Broadcasting System

It exposes global events for:

- Update
- FixedUpdate
- LateUpdate

Any script can subscribe to these instead of using its own update loops.

---

2. Gizmo Rendering System

It manages a list of collision sensor objects and:

- Draws debug wireframe boxes in the editor
- Controls when and how they appear

CORE CONCEPT: Singleton Instance

Instance

Purpose

Stores the single global instance of this system.

Role:

- Ensures only one broadcaster exists
- Prevents duplicates across scenes

👉 This is the central access point for the system.

STATIC EVENTS (GLOBAL SYSTEM)

These allow other scripts to react to Unity lifecycle events globally.

OnUpdate

Purpose

Triggered every frame.

Usage:

- Movement systems
- UI updates
- Input handling

OnFixedUpdate

Purpose

Triggered at physics intervals.

Usage:

- Physics movement
- Rigidbody calculations
- Collision logic

OnLateUpdate

Purpose

Triggered after Update.

Usage:

- Camera follow systems
- Final adjustments after movement

👉 Instead of each script using its own update method, they can subscribe here.

SENSOR LIST SYSTEM

registeredSensors

Purpose

Stores all active collision sensors.

Role:

- Tracks objects that need gizmo visualization
- Centralized debug rendering system

👉 Only sensors registered here will be drawn.

UNITY LIFECYCLE METHODS

Awake

This sets up the singleton behavior.

Step 1: Check for duplicates

If another instance already exists:

- Destroy this object

👉 Prevents multiple broadcasters

Step 2: Assign instance

Stores this object as the global instance.

Step 3: Persistence

Marks object as:

- Not destroyed when loading new scenes

👉 Keeps system active across the entire game

Update

Broadcasts global update event.

Behavior:

- Invokes OnUpdate event if any subscribers exist

FixedUpdate

Broadcasts physics update event.

Behavior:

- Invokes OnFixedUpdate event

LateUpdate

Broadcasts late update event.

Behavior:

- Invokes OnLateUpdate event

👉 These replace traditional per-script update loops if used correctly.

GIZMO SYSTEM (EDITOR VISUALIZATION)

OnDrawGizmos

This method runs only in the editor.

It is responsible for drawing debug visuals.

Step 1: Loop sensors

Iterates through all registered collision sensors.

Step 2: Get sensor settings

Each sensor provides:

- Gizmo mode (always, selected only, none)
- Target object reference
- Color
- Shape data (position, size, rotation)

Step 3: Always mode

If sensor is set to ALWAYS:

- Draw gizmo immediately

Step 4: Selected-only mode (Editor only)

If in Unity Editor:

- Gizmo is only drawn when:
    - Object is selected in hierarchy

👉 Helps reduce visual clutter

DrawSensorGizmo

This method actually renders the box.

Step 1: Set color

Uses sensor-defined color or default red.

Step 2: Apply transform matrix

Defines:

- Position
- Rotation
- Scale

👉 This ensures correct world alignment

Step 3: Draw cube

Renders wireframe box representing collision volume.

SENSOR MANAGEMENT METHODS

AddSensor

Purpose

Registers a sensor to be drawn.

Behavior:

- Adds sensor to list if not already present

👉 Prevents duplicates

RemoveSensor

Purpose

Unregisters a sensor.

Behavior:

- Removes sensor from list if exists

👉 This allows sensors to dynamically appear and disappear.

INITIALIZATION SYSTEM

Initialize

This ensures the system exists automatically.

Step 1: Check instance

If already exists:

- Do nothing

Step 2: Create object

Creates a hidden GameObject:

- Named “[Physics Update Broadcaster]”
- Adds this component

Step 3: Persistence

Marks object as:

- Not destroyed between scenes

👉 Guarantees system always exists

HOW THE SYSTEM WORKS (FLOW)

Runtime event flow:

1. Game starts
2. Initialize ensures singleton exists
3. Awake enforces only one instance
4. Every frame:
    - Update → OnUpdate event fires
    - FixedUpdate → OnFixedUpdate fires
    - LateUpdate → OnLateUpdate fires

Sensor system flow:

1. Sensor registers itself
2. Adds itself to list
3. OnDrawGizmos runs in editor
4. Each sensor is evaluated
5. Gizmos drawn depending on mode

USAGE

This system is used for:

---

1. Global event system

- Replace multiple Update methods
- Centralized update handling

---

2. Physics systems

- Unified physics update callbacks

---

3. Debug visualization

- Show collision boxes
- Debug sensor ranges

---

4. Modular architecture

- Systems subscribe instead of directly updating

---

5. Editor tools

- Visual debugging in Scene view

SUMMARY

- Instance ensures single global system
- OnUpdate / OnFixedUpdate / OnLateUpdate provide global events
- registeredSensors tracks debug collision objects

- OnDrawGizmos renders editor visuals
- Add/RemoveSensor manages dynamic registration
- Initialize guarantees automatic setup

Core loop:

1. System initializes automatically
2. Scripts subscribe to global events
3. PhysicsUpdateBroadcaster broadcasts lifecycle events
4. Sensors are registered and visualized
5. Editor shows gizmos based on rules

Action Block Timer

This script is a **small utility timer used to temporarily block an action**, such as jumping, crouching, attacking, or any gameplay input that should have a short cooldown before it can be used again.

Instead of being tied to Unity components, it is a **self-contained logic class** that can be created, updated, and checked by any system.

Overall Idea

The system works like a simple countdown lock:

1. You activate the block
2. A timer starts counting down
3. While the timer is active, the action is “locked”
4. When time runs out, the action becomes available again

CLASS: ActionBlockTimer

This is a reusable utility that:

- Stores a cooldown duration
- Tracks remaining block time
- Exposes whether the action is currently blocked
- Provides manual control over activation and reset

PROPERTY

IsBlocked

Purpose

Indicates whether the action is currently blocked.

Role:

- true → action cannot be used
- false → action is allowed

Behavior:

- Read-only from outside
- Only changed internally by the timer system

👉 This is the main check used by gameplay code.

PRIVATE FIELDS

---

timer

Purpose

Stores remaining time before the block ends.

Role:

- Counts down every frame
- Determines when blocking ends

---

duration

Purpose

Defines how long the block lasts.

Role:

- Fixed value set when object is created
- Used each time the block is activated

👉 This is the cooldown length.

CONSTRUCTOR

ActionBlockTimer(duration)

Purpose

Creates a new timer instance with a defined block duration.

Step-by-step behavior:

1. Stores duration value
2. Sets IsBlocked to false initially
3. Resets timer to zero

👉 At creation:

- Nothing is blocked yet
- It is ready to be activated

METHODS

Activate

Purpose

Starts the blocking process.

Behavior:

1. Sets IsBlocked to true
2. Resets timer to full duration

Result:

- Action becomes temporarily disabled
- Countdown begins immediately

Update

Purpose

Updates the countdown (must be called every frame).

Behavior step-by-step:

1. Check if blocked

If IsBlocked is false:

- Do nothing

👉 Saves performance

2. Reduce timer

Subtracts elapsed frame time from timer.

This means:

- Timer decreases smoothly in real time

3. Check expiration

If timer reaches zero or below:

- IsBlocked becomes false

👉 Action is now available again

Reset

Purpose

Immediately cancels the block.

Behavior:

1. Sets IsBlocked to false

2. Resets timer to zero

Result:

- Action becomes instantly usable again
- No waiting required

HOW THE SYSTEM WORKS (FLOW)

Step 1: Creation

A timer is created with a duration:

- Example: 0.15 seconds

At this point:

- Not blocked yet

Step 2: Activation

When an action is used:

- Activate is called
- Timer starts counting down

Step 3: Frame updates

Every frame:

- Update reduces timer
- Block remains active

Step 4: Expiration

When timer reaches zero:

- IsBlocked becomes false
- Action is allowed again

Step 5: Optional reset

At any time:

- Reset can cancel the block early

USAGE

This utility is commonly used for:

1. Input buffering control

- Prevent double jumping
- Avoid rapid crouch toggles

2. Combat systems

- Attack cooldowns
- Prevent spam attacks

3. Movement systems

- Dash cooldowns
- Sprint toggles

4. Animation safety

- Prevent overlapping transitions

5. Gameplay pacing

- Adds delay between repeated actions

EXAMPLE BEHAVIOR

If used for crouching:

1. Player presses crouch
2. Activate is called
3. Crouch becomes blocked
4. Timer counts down (0.15s)
5. After time passes:
    - crouch becomes usable again

WHY THIS IS USEFUL

1. Lightweight

- No MonoBehaviour required
- Works anywhere

2. Reusable

- Can be used for any action

3. Predictable timing

- Uses delta time for smooth countdown

4. Easy integration

- Just call Activate + Update + IsBlocked

5. Clean gameplay control

- Prevents input spam issues

SUMMARY

- duration defines block length
- timer counts down remaining time
- IsBlocked controls whether action is allowed
- Activate starts cooldown
- Update reduces time each frame
- Reset cancels block immediately

Core loop:

1. Activate block
2. Countdown begins
3. Update reduces timer each frame
4. When timer ends → unblock
5. Optional reset can override anytime

Box Collision Sensor

This script is a **runtime 3D box collision detection system** built in a very modular way. Instead of being tied to a specific character or object, it works through "providers" (functions that supply values dynamically), and it also plugs into a global update system for consistent physics checks and editor visualization.

🧠 Overall Purpose

This system:

- Checks for collisions inside a box volume (like a hitbox or sensor area).
- Lets you define *how* the box behaves at runtime (position, size, filters, etc.).
- Supports filtering (ignore tags, ignore children, etc.).
- Can log collisions for debugging.
- Can draw gizmos in the editor for visualization.
- Registers itself into a global update broadcaster so it runs automatically.

🔌 Core Concept: "Providers"

Instead of storing fixed values, the script uses **providers**.

A provider is basically:

"a function that returns a value when asked"

So instead of storing:

- position
- size
- rotation
- layer mask

It *asks another system every frame*.

Why this matters:

- The collision box can move dynamically.

- It can depend on animations, player state, or logic.
- It stays reusable for any system.

Main Variables (Providers Section)

These are the **inputs that define how collision detection behaves**.

Enable Detection Provider

Controls if collision checks should run at all.

- If false → sensor does nothing.
- Useful for disabling during cutscenes or menus.

Debug Log Provider

Controls whether collisions print messages in the console.

- True → logs every detected collider.
- False → silent mode.

Filter Mode Provider

Defines how results are filtered after detection.

Possible behaviors:

- no filtering
- ignore children
- ignore tags
- both combined

Reference Parent Transform Provider

Used when filtering by hierarchy.

- If set → ignores objects that are children of this transform.
- Commonly used to avoid self-collision.

Ignored Tags Provider

A list of tags to ignore.

Example usage:

- ignore “Player”
- ignore “Weapon”
- ignore “TriggerZone”

🔷 Collision Layer Mask Provider

Defines which layers can be detected.

- Works like a mask filter for physics layers.
- Helps reduce unnecessary checks.

🔷 Trigger Interaction Provider

Defines whether trigger colliders are included or ignored.

🔷 Box Shape Providers (Core Geometry)

These define the collision box:

- **Box Center Provider** → world position of the box
- **Box Size Provider** → width, height, depth
- **Box Rotation Provider** → orientation of the box

These make the detection volume dynamic.

🔷 Gizmo Providers (Editor Visualization)

These are only for editor visualization:

- **Gizmo Target Object Provider** → object used for selection checks
- **Gizmo Drawing Mode Provider** → when gizmos should appear:
  - never
  - always
  - only when selected
- **Gizmo Color Provider** → color of the box in editor

 Internal Fields

🔶 hitResults (Collider array)

A reusable fixed-size array that stores collision results.

Why it exists:

- avoids memory allocation every frame
- improves performance

It can store up to 16 colliders per check.

🔶 collisionDetected

A simple boolean flag:

- true → at least one valid collision detected
- false → no valid collision

This is what external systems usually read.

🎬 Constructor (Setup Phase)

When the sensor is created, it:

1. Stores all providers

Each input function is saved for later use.

2. Assigns defaults if missing

If something is not provided:

- identity rotation is used
- full layer mask is used
- triggers are ignored
- detection is enabled by default

3. Registers to update system

It connects to:

- a global FixedUpdate event
- a global sensor registry (for gizmos)

So it automatically starts working without needing a MonoBehaviour.

🧹 Dispose Method

This removes the sensor completely.

It:

- unsubscribes from update loop
- removes itself from gizmo system
- clears the reference

Used when:

- destroying an enemy
- unloading a system
- cleaning up memory

⚙️ Core Logic: Collision Check

This is the heart of the system.

Step 1 — Check if enabled

If detection is off → exit immediately.

Step 2 — Get runtime values

It requests current values from providers:

- center position
- size
- rotation
- layer mask
- trigger mode

This means values can change every frame.

Step 3 — Physics query

It performs a **box overlap check**:

“What colliders are inside this box right now?”

Results are stored in hitResults.

No memory allocation is created here (important optimization).

Step 4 — Filter results

Each collider is tested:

- Is it a child we should ignore?
- Does it have an ignored tag?
- Does it pass filter rules?

Only valid results count.

Step 5 — Debug (optional)

If enabled:

- logs collider name in console

Step 6 — Set result

If any valid collision exists:

- collisionDetected = true
- loop stops early

🧠 **Filtering Logic (Important Section)**

This decides if a collider is valid.

It supports 4 modes:

None

Everything is accepted.

IsNotChildOf

Rejects objects that are children of a given transform.

Used to avoid:

- self detection
- detecting parts of same character

IgnoreByTags

Rejects objects with specific tags.

All

Combines both:

- ignore children
- ignore tags

🎨 Gizmo System (Editor Visualization)

This runs in editor only.

For every registered sensor:

If mode = Always

→ draw box

If mode = SelectedOnly

→ draw only when selected in editor

Then it draws:

- wireframe cube
- correct position
- correct rotation
- correct size
- correct color

This helps debugging collision areas visually.

📌 How You Use This System

Typical usage flow:

1. Create sensor

You define:

- position provider
- size provider

- filters
- layer mask

2. Attach to a system

It automatically:

- registers itself
- starts updating

---

3. Read result

You check:

- collisionDetected

Example usage:

- block player attack if true
- trigger damage
- activate pressure plate
- detect walls ahead

---

4. Dispose when done

Remove sensor when object is destroyed.

🧩 **Big Picture**

This script is basically a:

high-performance, modular, runtime-configurable hitbox system

Instead of being a simple collider check, it behaves like:

- a configurable sensor engine
- driven by dynamic data sources
- integrated into a global update system
- optimized for minimal allocations
- fully editor-debuggable

Check Valid Angle Sensors
This script is a **ground-slope detection and friction control system** used for character movement physics. It continuously checks what kind of surface the player is standing on, measures its slope angle, and then decides whether the ground is stable or too steep. Based on

that, it also changes the physics material applied to the player's colliders (affecting sliding, grip, and movement behavior).

It runs automatically through a shared physics update system instead of being attached directly to a Unity component.

🧠 Core Idea

Every physics tick, the system:

1. Shoots a ray straight downward from the player.
2. Detects what it hits.
3. Measures the slope angle of that surface.
4. Decides if the surface is walkable.
5. Applies the correct friction material.
6. Stores whether the ground is valid or not.

🔌 Provider Variables (Dynamic Inputs)

These are functions that supply live data every time the system runs.

They make the system flexible and reusable.

🎯 targetTransform

Provides the player's transform (position and orientation).

Used as:

- the origin of the downward raycast

📐 maxStableAngle

Defines the maximum slope angle that is considered walkable.

Example:

- 45° → normal slopes are allowed
- above 45° → considered too steep

If not provided, default is **45 degrees**.

---

raycastDistance

Defines how far downward the system checks for ground.

If not provided:

- default = 0.5 units

---

highFrictionMaterial

Material applied when the ground is stable.

Effect:

- more grip
- less sliding
- better control

---

lowFrictionMaterial

Material applied when the ground is too steep.

Effect:

- more sliding
- less control
- slippery behavior

---

colliders

Provides all player colliders that will receive the friction material.

This allows:

- multiple colliders on a character
- consistent physics behavior

---

debug

Enables or disables debug output:

- ray visualization
- console logs

- warnings

📦 Internal Fields

✅ isValidAngle

This is the main output of the system.

It tells:

- true → player is on a valid slope
- false → slope is too steep or invalid ground

Other systems can use this for:

- movement restrictions
- jumping rules
- animation changes

📊 raycastHits

A reusable array that stores raycast results.

Why it exists:

- avoids memory allocations
- improves performance
- supports up to 8 hits per check

🏭 Constructor (Setup Phase)

When the system is created, it:

1. Stores all providers

So it can request values dynamically every physics tick.

---

2. Sets default values

If optional values are missing:

- max slope → 45°
- ray distance → 0.5

- debug → false

3. Registers to physics update loop

It connects to a global system:

- runs every FixedUpdate

So it behaves like a physics component without being one.

🧹 **Dispose Method**

Used to clean up the system.

It:

- removes it from update loop
- clears reference

Prevents:

- memory leaks
- duplicate updates

⚙️ **Core Logic: CheckRaycast**

This is the main function that runs every physics tick.

🟢 **Step 1 — Get runtime values**

It retrieves:

- player position
- ray distance
- slope limit

🟢 **Step 2 — Define ray origin**

The ray starts slightly above the player:

- prevents hitting the player itself

- ensures clean ground detection

🟢 Step 3 — Shoot ray downward

A physics raycast is performed:

“What is directly under the player?”

It uses a non-alloc method:

- no garbage creation
- optimized for performance

🔴 Step 4 — No ground found

If nothing is hit:

- mark slope as invalid
- optionally draw red debug ray
- exit early

🟡 Step 5 — Sort results

If multiple hits exist:

- closest surface is processed first

This ensures correct ground detection.

🧪 Step 6 — Filter each hit

Each collision is tested:

Skip if:

- collider is part of the player itself
- surface is too vertical (wall-like)
- invalid geometry

📐 Step 7 — Calculate slope angle

The angle is computed between:

- surface normal
- world up direction

Meaning:

- flat ground → small angle
- steep slope → large angle

🟢 Step 8 — Decide stability

If slope angle is within limit:

- stable ground

Otherwise:

- unstable slope

A small tolerance (0.1°) is added for smooth behavior.

🧱 Step 9 — Apply friction material

Depending on slope:

- stable → high friction material
- unstable → low friction material

This directly affects movement physics:

- sliding
- grip
- acceleration behavior

🐞 Step 10 — Debug visuals (optional)

If enabled:

- green ray = stable ground
- yellow ray = unstable slope
- red ray = no ground

Also prints:

- collider name
- angle value

- warnings for steep slopes

🧠 Step 11 — Final result

The system:

- sets isValidAngle
- exits immediately after first valid surface

❌ Step 12 — No valid surfaces found

If all hits were ignored:

- treats ground as invalid
- logs warning (if debug enabled)

🧩 RaycastHitComparer (Utility Class)

This is a helper used to sort raycast results.

It ensures:

- closest surface is processed first

Why this matters:

- avoids detecting far-away surfaces incorrectly
- improves accuracy of ground detection

🎮 How This Is Used in Gameplay

This system typically controls:

🏃 Movement systems

- prevent movement on steep slopes
- reduce speed on slopes

🧍 Physics behavior

- sliding on steep ground
- stable walking on flat ground

🎥 Animation logic

- slope-based movement animations

🎮 Gameplay rules

- disable jumping on walls
- detect ground validity

🧠 Big Picture

This script is essentially:

A real-time slope analyzer + friction controller

It combines:

- raycasting
- slope math
- physics material swapping
- performance optimization
- debug visualization
- global update integration

Custom Inspector Base

This script is a **foundation system for building custom Unity Inspectors**. Instead of writing repetitive editor UI code every time, it provides a structured base that handles common tasks like property drawing, foldouts, and safe serialization handling.

It is designed to be inherited by other custom inspectors so they can focus only on layout logic.

🧠 Overall Purpose

This system:

- Standardizes custom inspector creation.
- Handles Unity serialization safely.
- Provides reusable UI helpers.
- Adds foldable UI groups with saved state.
- Makes property access safer and easier to maintain.

📦 Core Concept

Instead of writing raw inspector code repeatedly, derived inspectors only implement:

“What should the inspector look like?”

While this base class handles:

- drawing lifecycle
- property synchronization
- foldouts
- caching
- error handling

📦 Main Variables

🔷 foldoutStates (static dictionary)

This stores the open/closed state of UI groups.

What it does:

- remembers if a foldout is expanded
- prevents repeated EditorPrefs access
- shared across all inspectors using this base class

Why it matters:

Without this:

- foldouts would reset constantly
- performance would be worse

🔷 script (typed target reference)

This is a strongly-typed version of the inspected object.

What it gives:

- direct access to the target component
- avoids repeated casting
- makes derived classes cleaner

⚙️ Lifecycle Methods

🔄 OnEnable

Runs when inspector becomes active.

What it does:

- assigns the inspected object to script
- prepares access to the target component

🖼 OnInspectorGUI

This is the main inspector loop.

Step-by-step behavior:

1. Sync data

Pulls latest values from Unity serialization system.

---

2. Adds spacing

Creates visual separation in the inspector.

---

3. Locks script field

Shows the script reference but makes it non-editable.

---

4. Calls DrawInspector

This is the key abstraction point.

👉 Derived classes define UI here.

---

5. Applies changes

Writes modified values back into Unity.

🧩 Core Abstract Method

🔷 DrawInspector

This is the only method derived classes must implement.

Purpose:

Defines how the inspector UI looks.

Why it exists:

- enforces structure
- separates logic from layout
- keeps code reusable

🧰 Property Drawing System

This section simplifies working with Unity serialized fields.

🔷 GUIProperty (string version)

Draws a property using its name.

Steps:

1. finds property inside serialized object
2. validates it exists
3. draws it in inspector

If not found:

- shows error message in inspector

🔷 GUIProperty (expression version)

Same function but safer.

Instead of using text:

- uses actual field reference

Benefits:

- avoids typos
- works with refactoring
- compiler helps catch errors

🔷 GetPropertyName

Extracts the field name from a lambda expression.

Handles two cases:

1. Direct access

Example:

- field reference

2. Value types (boxed)

Handles internal conversion cases automatically.

If invalid:

- throws an error

📂 **Foldout Group System**

This system creates collapsible inspector sections.

🔷 **DrawGroup**

This builds a styled foldout section.

Inputs:

- group name (label + tooltip)
- content drawing function

What it does step-by-step:

1. Gets saved state

Checks if group is open or closed.

---

2. Creates header layout

Defines UI space for the foldout.

---

3. Forces full width alignment

Fixes Unity inspector layout inconsistencies.

4. Draws visuals

Adds:

- top border line
- subtle background

5. Creates foldout label

Styled version with bold text.

6. Displays foldout UI

User can expand or collapse group.

7. Saves state if changed

If user toggles it:

- updates memory
- saves to EditorPrefs

8. Draws content if open

If expanded:

- increases indentation
- draws inner UI
- reduces indentation back

💾 **Persistence System**

These methods manage foldout memory.

🔷 **GetGroupState**

Retrieves foldout state.

Priority order:

1. in-memory cache (fast)
2. EditorPrefs (persistent storage)

If not found:

- defaults to “open”

🔷 SaveGroupState

Stores foldout state.

It does two things:

- updates cache
- saves to Unity EditorPrefs

So state survives:

- editor restart
- project reload

🔷 GetPrefsKey

Creates a unique identifier per group.

Example format:

- “HeaderGroupGUI_Foldout_MyGroup”

This prevents conflicts between inspectors.

🧠 How You Use This Script

You never use it directly.

Instead, you:

1. Create a new inspector class

That inherits this base system.

---

2. Implement DrawInspector

Inside it you:

- define layout
- call GUIProperty
- create groups with DrawGroup

---

3. Build structured inspector UI

Example usage patterns:

- foldable sections for settings
- clean property layouts
- reusable inspector patterns

🎮 **Why This System Is Useful**

Without it:

- inspectors are repetitive
- hard to maintain
- error-prone
- inconsistent UI

With it:

- structured UI design
- reusable components
- safer property access
- persistent foldout states
- cleaner editor code

🧩 **Big Picture**

This script acts like:

A **framework layer for Unity custom inspectors**

It doesn't define UI itself — it defines how UI should be built safely, consistently, and efficiently.

Custom Player Data
This utility is a **data save/load system for player information**, designed to convert a flexible dictionary of values into JSON and restore it later while preserving type information (including Unity types and enums).

It is built around two main ideas:

- Everything is stored as a **string + type metadata**
- Everything is reconstructed using that type metadata when loading

---

🔷 Overall Purpose

The class lets you:

- Save player data like health, position, settings, etc.
- Store it as JSON
- Reload it later with correct types restored

It supports:

- Primitive types (int, float, bool, string)
- Unity types (Vector2, Vector3, Quaternion, etc.)
- Enums
- Custom objects (fallback via string conversion)

📦 Internal Data Structure

Entry (single data item)

Each stored value becomes an “entry” with 3 parts:

- **key**
  - The name of the data (example: "Health" or "Speed")
- **value**
  - The actual stored data, always converted into text
  - Example: "100", "true", or JSON text for Unity objects
- **type**
  - The full type name of the original value
  - Used later to rebuild the correct type when loading

👉 Think of it like:

“Here is the value, and here is what it originally was”

EntryList (container)

This is just a wrapper that holds:

- A list of all entries

It exists so Unity can easily convert everything into JSON using its serializer.

💾 SaveData (Serialization)

Usage:

This method converts a dictionary into a JSON string.

Input:

- A function that returns a dictionary of data

Example usage idea:

- Health = 100
- Speed = 5.5
- State = Running

How it works step-by-step:

1. Build dictionary

It calls the provided function and gets all player data.

2. Create empty storage list

A container is created to hold all entries.

3. Loop through each value

For every key-value pair:

- If value is null → skip it
- Detect the type of the value

4. Convert value into string

Different rules depending on type:

- **Enum**
  - Stored as text name
  - Example: "Running"
- **Unity types**
  - Converted to JSON format
- **Everything else**

- Converted using normal text representation

5. Store metadata

Each entry saves:

- key
- converted value
- original type name

6. Convert everything to JSON

Final step:

- The whole list becomes a JSON string

👉 Output:
A single JSON blob representing all player data

📥 **LoadData (Deserialization)**

Usage:

Takes JSON and rebuilds the original dictionary.

How it works step-by-step:

1. Read JSON

The JSON is converted back into:

- a list of entries

2. Create empty dictionary

A new dictionary is prepared to receive reconstructed values.

3. Conversion table (important part)

A set of known converters is defined for common types:

- float → parse number
- int → parse integer
- bool → parse true/false
- string → direct value
- Vector2/3/4 → JSON conversion
- Quaternion → JSON conversion

👉 This avoids manual parsing everywhere

---

4. Process each entry

For each saved item:

a) Recover type

It reads the stored type name and tries to locate it again.

If type is missing:

- Warning is shown
- Entry is skipped

---

b) Convert value back

Three possible paths:

- **Enum**
  - Rebuilt from text name
- **Known type**
  - Uses predefined converter
- **Unknown type**
  - Tries generic conversion fallback

---

5. Store reconstructed value

Each key is restored into the dictionary with correct type.

---

6. Return result

Finally:

- The dictionary is passed into a callback function
- The system that requested loading receives fully restored data

⚙️ Usage Example (Conceptual)

Saving:

- You provide a function that returns player data
- The system returns a JSON string

Loading:

- You pass the JSON string
- You receive a rebuilt dictionary with correct types

🎯 Key Design Idea

This system is powerful because:

✓ It does not require predefined save structures
✓ It works with different data types automatically
✓ It preserves type information safely
✓ It is extensible (new types can be added later)

⚠️ Important Limitations

- Reflection-based type reconstruction can fail if types are renamed or missing
- Complex custom classes may not serialize correctly without extra logic
- Conversion fallback may lose precision in some cases

Mono Behaviour Extensions

This script is a **reflection-based bridge between unknown player controllers and systems that need movement state data**. It allows any MonoBehaviour to be queried dynamically for properties like grounded, moving, crouching, and running—without needing a strict inheritance relationship.

It is split into two main parts:

- A **generic reflection extension system**
- A **movement-state adapter built on top of it**

🔧 1. MonoBehaviourExtensions (Reflection Utility)

This is a static helper that extends any Unity component behavior with dynamic property reading.

🔷 Purpose

It allows code to ask:

“Does this object have a property called X, and can I read it as type Y?”

Without needing compile-time knowledge of the script.

🧠 TryGetProperty (main method)

What it does

Tries to read a public property from a component using its name.

Inputs

- **mono**
  - The object being inspected (any Unity component)
- **propName**
  - The name of the property you want to access (as text)
- **value (output)**
  - Where the result is stored if successful

Output

- Returns:
  - true → property exists and is correct type
  - false → property missing or wrong type

⚙️ Internal process

1. Search property by name

It looks at the object’s type and tries to find a public property matching the given name.

---

2. Validate type

It checks:

- Does the property exist?

- Is it the expected type?

If both are true → safe to read

---

3. Read value

If valid:

- It extracts the value dynamically at runtime

---

4. Failure case

If anything is wrong:

- Returns default value (like false, 0, null)
- Signals failure

🎯 **Usage idea**

This allows:

- Reading unknown scripts safely
- Decoupling systems (no direct dependency on player controller classes)
- Flexible architecture for modular controllers

🧩 **2. IPlayerMovementState (Interface)**

🔷 **Purpose**

Defines a **contract of required movement properties**.

Any system that implements this “shape” must expose:

- IsGrounded
- IsMoving
- IsCrouching
- IsRunning

🧠 **Important idea**

Even though reflection is used, this interface defines:

“What movement data is expected in the system”

It acts like a blueprint for consistent behavior.

🧠 3. PlayerMovementStateAdapter (Core Bridge)

This is the main adapter that connects:

- unknown MonoBehaviour scripts
  → to a clean movement state interface

🔷 Purpose

It allows any controller to behave as if it implements movement state,
even if it actually does not.

It does this using reflection.

🔧 Internal variable

target

- The MonoBehaviour being inspected
- This is the source of all movement data

⚙️ Constructor

What it does

Stores the target object so it can be queried later.

Usage

You create it like:

- “Take this controller and let me read movement info from it”

📊 Movement Properties (IsGrounded, IsMoving, etc.)

Each of these works the same way:

Example: IsGrounded

What it does

Checks if the target object has a property called:

- "IsGrounded"

If yes:

- Returns its value

If no:

- Returns false

Internal logic per property

Each property:

1. Calls reflection helper

Uses the extension method to attempt retrieval.

2. Checks result

- If found AND true → returns true
- Otherwise → returns false

Why this pattern matters

It ensures:

- Safe access (no crashes if missing)
- Works with any controller script
- No hard dependency on specific player code

🎮 **Usage in practice**

This system is used when:

- You want multiple player controllers to work with the same systems
- You don't want tight coupling between movement and features (like sound, animation, UI)

Example scenario

A system like:

- footstep audio
- animation controller
- stamina UI

can simply do:

"Give me movement state"

without knowing what script provides it.

🧠 **Key Design Idea**

This script is built around:

✓ Reflection flexibility

Works with any script at runtime

✓ Interface abstraction

Defines expected behavior

✓ Adapter pattern

Bridges unknown scripts into a standard format

⚠️ **Trade-offs**

Pros:

- Extremely flexible
- No hard dependencies
- Works with modular systems

Cons:

- Slight performance cost (reflection)
- Errors only appear at runtime if property names mismatch

- Requires strict naming conventions

📜 **Summary**

- **Extension method** → safely reads properties by name
- **Interface** → defines expected movement data
- **Adapter** → connects unknown scripts to that interface

Together, they allow:

Any player controller → to behave like a standardized movement system without inheritance

On Input System Event

This script is a **generic input event dispatcher built on top of Unity's Input System**, designed to let multiple scripts react to the same input action independently, without interfering with each other.

It introduces a **layer between InputAction and gameplay scripts**, adding:

- hold detection
- press detection
- release detection
- per-owner isolation
- enable/disable control per listener
- safe multi-subscriber handling

🧠 **Core Idea**

Normally, an input action is shared globally.

This system changes that by saying:

"Multiple scripts can listen to the same input, but each one behaves independently based on its own rules."

🧩 **1. OnInputSystemEvent (Main Dispatcher)**

This is the **central static manager** that tracks all input actions.

🔷 **Main variable: states**

- Stores every InputAction being tracked
- Each action has a "State" object attached

👉 Think of it like:

InputAction → its own runtime brain

🧠 2. State (per InputAction runtime memory)

Each InputAction gets its own State object.

This is where all logic happens.

🔷 Key variables inside State

action

- The actual InputAction being monitored

---

lastValue

- Stores last input value read (float, Vector2, bool, etc.)
- Used to detect changes over time

OnHold / OnPressed / OnReleased

Each is a list of callbacks:

- **owner**
  - The script that registered the input (important for separation)
- **cb**
  - Function to call when input event happens
- **enabled**
  - Condition that decides if this owner is allowed to receive input

👉 This allows:

- multiple listeners per input
- independent enable/disable logic per listener

---

ownerWasHeld

- Tracks whether each owner was holding the input in the previous frame
- Used to detect transitions like:
  - pressed → holding

- holding → released

🔁 **State.Update (core runtime loop)**

This is the **main processing function**, executed every input update cycle.

Step-by-step behavior:

1. Safety checks

If:

- action does not exist
- or is disabled

→ stop processing

2. Read current input value

Gets current value from InputAction:

- float, Vector2, bool, etc.

Then converts it into:

- “is this input active?”

(using IsNonZero logic)

3. Collect all owners

Builds a list of all scripts listening to this input.

4. Process each owner separately

Each owner is handled independently:

🔷 **Step A — Check if owner is enabled**

Each owner has an optional condition:

- if false:
  - simulate release event (if needed)
  - skip processing

👉 This allows things like:

disable input when UI is open

🔷 **Step B — Handle input states**

Depending on input:

If input is held:

- If owner was NOT holding before:
  → trigger “Pressed”
- Always trigger “Hold”

If input is NOT held:

- If owner WAS holding before:
  → trigger “Released”

Then update tracking state

⚙️ **IsNonZero (input activity detection)**

This function decides if input is “active”.

It supports multiple types:

- float → checks if value is above small threshold
- Vector2 / Vector3 → checks magnitude
- Quaternion → checks if not identity
- bool → direct value
- fallback → compares with default

👉 Purpose:

Normalize different input types into a single “active or not” rule

---

🧹 UnregisterAll (cleanup per owner)

Removes:

- all hold callbacks
- all pressed callbacks
- all released callbacks
- tracking data

👉 Used when:

- object is destroyed
- input is no longer needed

🎮 3. Registration Methods (API layer)

These are the public ways to subscribe to input.

🔷 RegisterHold

Calls repeatedly while input is active.

Usage:

movement, camera rotation, continuous actions

🔷 RegisterPressed

Calls once when input starts.

Usage:

jump, attack, interact

🔷 RegisterReleased

Calls once when input stops.

Usage:

stop sprint, stop charging, end hold action

🔷 **UnregisterAll**

Removes all callbacks for a specific owner + action.

🔷 **Clear**

Removes everything for that action completely.

🔗 **4. WithAction (Fluent API builder)**

This is the **user-friendly entry point**.

What it does:

- Enables the InputAction
- Creates or gets its State
- Returns a configuration object

👉 Purpose:

Start chaining input setup cleanly

📦 **5. OnInputSystemEventConfig (Fluent wrapper)**

This is the **builder that makes input setup readable and chained**

🔷 **Stored variables**

- action → input being configured
- owner → script using it
- enabled → condition for activation

🔷 **Methods**

OnHold(callback)

Registers continuous input behavior

OnPressed(callback)

Registers single activation event

OnReleased(callback)

Registers release event

Dispose()

Removes all callbacks for this owner

🎯 Usage Example (conceptual)

A script might do:

- bind input
- listen for press
- listen for hold
- listen for release
- auto-clean when destroyed

All in a single chain.

🧠 Why this system is powerful

✓ Multi-script safe

Multiple scripts can use same input safely

✓ Ownership system

Each script is isolated

✓ Flexible input types

Works with float, Vector2, bool, etc.

✓ **Clean lifecycle handling**

Automatically handles press/hold/release transitions

✓ **Centralized update**

No need for each script to poll input individually

⚠ **Trade-offs**

- More complex than direct InputAction usage
- Slight overhead due to dictionary + iteration
- Requires strict lifecycle management (Dispose is important)

📜 **Summary**

This system turns Unity input into:

A **shared event system with per-script isolation and full lifecycle control**

It works by:

- tracking input actions centrally
- assigning per-action runtime state
- isolating listeners by owner
- converting raw input into press/hold/release events

Player Jump Controller

This script is a **modular jump system for a Rigidbody-based character**, designed to be flexible and reusable. Instead of directly depending on a specific player controller, it uses **functions (providers)** to fetch data at runtime, which makes it highly decoupled and adaptable.

🧠 **General Idea**

The system handles:

- Standard jumping
- Multi-jumping (like double jump)
- Coyote time (jumping shortly after leaving a platform)
- Jump cooldown (prevents false ground detection immediately after jumping)

It runs automatically every physics step and reacts when a jump is requested.

📦 Fields (What each variable does)

📎 Runtime provider functions (inputs from outside)

These are functions that supply real-time data:

- **targetRigidbody**
  → Provides the Rigidbody that receives the jump force.
- **isGrounded**
  → Returns whether the character is currently touching the ground.
- **maxJumps**
  → Defines how many jumps are allowed before needing to land again (default is 1).
- **coyoteTime**
  → Defines how long after leaving the ground the player can still jump (grace window).

These make the system independent of any specific player script.

📊 Internal state tracking

- **jumpCount**
  → Tracks how many jumps were already used since last landing.
- **lastGroundedTime**
  → Stores the last moment the character touched the ground.
- **wasGroundedLastFrame**
  → Used to detect transitions (landing or leaving ground).
- **jumpCooldownTime**
  → Prevents ground detection immediately after jumping (avoids false “grounded” states).
- **groundedIgnoreDurationAfterJump (constant)**
  → Fixed small delay (0.1 seconds) after jumping where ground detection is ignored.

⏳ Coyote system state

- **coyoteJumpUsed**
  → Ensures the “late jump” (coyote jump) is only used once per fall.
- **isInCoyoteWindow**
  → Indicates whether the player is still inside the coyote time period.

📦 Constructor (How it is created)

When this system is created:

- Required functions must be provided:
  - Rigidbody source
  - Ground check function

- Optional functions:
  - max jumps (defaults to 1)
  - coyote time (defaults to 0)

Then:

- The system subscribes itself to the physics update loop
- Meaning it automatically runs every FixedUpdate via the broadcaster

---

🎮 Public Methods

🟢 OnJump (main action)

This is the method you call when the player presses jump.

What it does step by step:

1. **Validation checks**
   - Ensures Rigidbody exists
   - Ensures game is not paused (time scale not zero)
2. **Reads current conditions**
   - Grounded state
   - Maximum allowed jumps
3. **Checks if jumping is allowed**
   Player can jump if:
   - They are on the ground
   - OR they are inside coyote time and haven't used it yet
   - OR they still have extra jumps available
4. **Applies jump**
   - Resets vertical velocity (prevents inconsistent jump height)
   - Applies upward impulse force
5. **Prevents immediate ground false detection**
   - Sets a short cooldown where ground detection is ignored
6. **Handles coyote logic**
   - Marks coyote jump as used if applicable
7. **Increases jump counter**
   - Tracks how many jumps were performed
8. **Returns result**
   - True if jump happened
   - False if blocked

🧹 Dispose (cleanup)

- Removes the system from the physics update loop
- Clears the reference so it can be garbage collected

Used when:

- Player is destroyed

- Scene changes
- System is no longer needed

🔄 Private Method: Update (core logic loop)

This runs every physics frame.

1. Reads grounded state

- Gets current grounded value

2. Applies jump cooldown rule

- If still inside post-jump cooldown → forces “not grounded”

---

3. Landing detection

When player goes:

- air → ground

Then:

- resets jump counter
- resets coyote usage
- stores landing time

---

4. Leaving ground detection

When player goes:

- ground → air

Then:

- starts coyote timer
- allows coyote jump again

---

5. Updates coyote window

- Checks if player is still within allowed grace time
- Based on time since last ground contact

---

6. Anti-exploit safety rule

If:

- player is in air
- coyote time ended
- no jump used yet

Then:

- consumes jump automatically

(This prevents unintended infinite “free jumps” from edge cases.)

🎯 How to use this system

You typically:

1. Create the controller

Provide:

- Rigidbody reference
- Ground check function
- Optional max jumps / coyote time

---

2. Call OnJump when input happens

Example usage conceptually:

- Player presses jump button → call OnJump()

---

3. Let Update run automatically

It is tied to physics updates, so:

- you do not manually update it
- it manages itself through the broadcaster

---

4. Dispose when needed

When player or scene ends:

- call Dispose to clean everything safely

Summary

This script is basically a **complete jump brain system** that:

- Decides when jumping is allowed
- Tracks jump state across frames
- Adds forgiving mechanics (coyote time)
- Prevents physics bugs after jumping
- Works independently from any specific controller

Player Move Controller

This script is a **Rigidbody-based movement controller** that applies horizontal movement (X and Z axes) while preserving vertical physics (Y axis). It is designed to be **decoupled**, meaning it does not directly depend on a player script, but instead receives data through functions.

General Idea

The system:

- Reads movement input direction (XZ plane)
- Applies velocity to a Rigidbody
- Preserves gravity and jumping (Y velocity untouched)
- Can optionally be blocked by obstacles
- Can be fully stopped or disposed safely

Fields (What each variable does)

External dependency functions

These are runtime “providers”:

- **targetRigidbody**
  → Supplies the Rigidbody that will receive movement.
- **speed**
  → Returns movement speed multiplier.
  → Default value is 3 if not provided.
- **isObstacle**
  → Returns whether movement should be blocked (for example: wall hit, stun, etc.).
  → Default is false if not provided.

Constructor (How it is created)

When the controller is created:

- Rigidbody provider is required
- Speed is optional (defaults to 3)
- Obstacle logic is optional (defaults to no blocking)

This makes the controller flexible and reusable across different characters.

🎮 Public Methods

🟢 OnMove (main movement function)

This is called whenever the player provides movement input.

What happens step by step:

1. Obstacle check

- If movement is blocked:
  - The system immediately calls Stop
  - No movement is applied

---

2. Gets Rigidbody

- Retrieves the target physics body
- If it doesn't exist → nothing happens

---

3. Reads speed

- Gets current movement speed value

---

4. Prepares movement

- Takes input direction (X and Z only)
- Normalizes it (ensures consistent speed in all directions)
- Multiplies by speed

---

5. Preserves vertical motion

- Reads current velocity
- Keeps Y velocity untouched (important for:
  - jumping
  - falling

- gravity)

6. Applies movement

- Sets new velocity:
  - X = horizontal movement
  - Y = unchanged vertical velocity
  - Z = horizontal movement

🛑 **OnStop (stop movement)**

Used when:

- player releases input
- obstacle blocks movement
- movement should instantly stop

What it does:

- Gets Rigidbody
- Reads current velocity
- Sets:
  - X = 0
  - Z = 0
  - Y = preserved

👉 Result: character stops sliding horizontally but still falls naturally

🧹 **Dispose (cleanup)**

- Simply clears the reference to the controller
- Used when:
  - character is destroyed
  - system is replaced
  - scene changes

🎯 **How to use this system**

1. Create controller

You provide:

- Rigidbody reference function
- Optional speed function

- Optional obstacle function

2. Feed input every frame

When player moves:

- Call OnMove with a 2D direction (X and Z)

Example conceptually:

- left/right = X
- forward/back = Z

3. Stop movement when needed

When input stops or blocked:

- Call OnStop

4. Dispose when done

When no longer needed:

- call Dispose

⚙ **Key design idea**

This system intentionally separates:

- **Input (direction)**
- **Physics (Rigidbody velocity)**
- **Rules (speed, obstacles)**

So it can be reused for:

- player characters
- NPCs
- AI movement
- physics-controlled entities

🧩 **Summary**

This script is a **clean velocity-based movement engine** that:

- Moves a Rigidbody using XZ input
- Keeps gravity and jumping intact
- Supports external speed control
- Supports movement blocking logic
- Can instantly stop horizontal motion
- Is fully modular and reusable

Player Rotation Controller

This script is a **smooth rotation controller for a 3D character**, designed to rotate a Transform toward movement input on the XZ plane. It uses physics-timed updates and smooth interpolation so the rotation feels natural instead of snapping instantly.

It is also fully decoupled: instead of directly depending on a player script, it receives data through functions.

🧠 General Idea

The system:

- Reads a 2D movement direction
- Converts it into a 3D direction (XZ plane)
- Rotates a Transform toward that direction
- Uses smooth interpolation (Lerp-style rotation)
- Only rotates when movement input is strong enough
- Runs automatically every physics tick

📦 Fields (What each variable does)

🔗 External provider functions

These are runtime inputs:

- **targetTransform**
  → The object that will rotate (usually the player model or root transform).
- **direction**
  → A 2D movement vector (X and Y, where Y represents Z in 3D space).
- **speed**
  → Controls how fast the rotation catches up to the target direction.
  → Default is 10 if not provided.
- **magnitude**
  → Minimum input strength required before rotation happens.
  → Prevents jitter when input is very small.

🏗️ Constructor (How it is created)

When the controller is created:

- It stores all provider functions
- Applies default values if speed or magnitude are not provided
- Automatically registers itself to the physics update loop

So it starts working immediately without needing manual update calls.

🎮 Public Methods

🧹 Dispose (cleanup)

Used when the system is no longer needed.

What it does:

- Unsubscribes from physics update loop
- Clears the reference

Used when:

- player is destroyed
- scene changes
- system is replaced

🔄 Core Logic: UpdateRotation (main behavior)

This runs every physics frame.

---

1. Reads input direction

- Gets the current 2D movement vector
- Example:
  - forward = (0, 1)
  - right = (1, 0)

---

2. Checks if input is strong enough

- Uses squared magnitude (faster than normal magnitude)
- If input is too small:
  - rotation is ignored

👉 This prevents:

- jittering
- unwanted micro-rotation when idle

---

3. Converts 2D to 3D direction

- Takes:
  - X → horizontal axis
  - Y → depth axis (Z in 3D world)

Result:

- A world direction vector on XZ plane

---

4. Calculates target rotation

- Creates a rotation that faces the movement direction
- Uses upward vector as “up axis”
- Ensures character stays upright

---

5. Smoothly applies rotation

Instead of snapping instantly:

- Current rotation gradually moves toward target rotation
- Speed is controlled by:
  - rotation speed
  - fixed delta time

👉 Result:

- smooth turning
- natural character movement
- no instant snapping

🎯 **How to use this system**

1. Create controller

Provide:

- Transform reference
- Movement direction source

- Optional speed and threshold

2. Feed movement input

Each frame:

- direction function returns current input

Example conceptually:

- WASD → direction vector
- joystick → direction vector

3. Let physics update handle rotation

No manual update needed:

- system auto-runs every FixedUpdate via broadcaster

4. Dispose when done

When character is destroyed or replaced:

- call Dispose

⚙ **Key design idea**

This system separates:

- **Input (direction)**
- **Logic (when to rotate)**
- **Presentation (smooth rotation of transform)**

So it can be reused for:

- players
- enemies
- NPCs
- automated agents

🧩 **Summary**

This script is a **smooth directional rotation system** that:

- Rotates a Transform based on movement input
- Works on the XZ plane (3D ground movement)
- Uses smooth interpolation for natural turning
- Ignores tiny inputs to prevent jitter
- Runs automatically on physics updates
- Is fully modular and reusable

Player Speed Control

This script is a **movement speed and stamina management system** that controls whether a character walks or runs, and manages stamina depletion and recovery over time. It is fully decoupled, meaning it does not directly depend on a player controller—it receives everything through function providers.

🧠 General Idea

The system:

- Switches between normal speed and running speed
- Uses stamina to limit running
- Drains stamina while running and moving
- Regenerates stamina when not running
- Automatically updates every physics frame
- Exposes values for UI and gameplay logic

📦 Fields (What each variable does)

🔗 Input provider functions (external data)

These functions supply runtime values:

- **normalSpeed**
  → Movement speed when walking.
- **fastSpeed**
  → Movement speed when running.
- **maxStamina**
  → Maximum stamina value (default 50).
- **minimumAmountVigor**
  → Minimum stamina required to allow running again after exhaustion (default 12.5).
- **staminaDepletionRate**
  → How fast stamina decreases while running.
- **staminaRecoveryRate**
  → How fast stamina regenerates when not running.
- **isMove**
  → Returns whether the player is currently moving.

⚙️ Internal state variables

- **useStamina**
  → Determines whether stamina system is active.
  → If false, running is always allowed without stamina rules.
- **currentStamina**
  → Current stamina value being tracked internally.
- **subscribedToUpdate**
  → Tracks whether the system is connected to the physics update loop.
- **isFast**
  → Indicates whether the player is currently trying to run.
- **haveEnoughStamina**
  → Indicates whether the player is allowed to run based on stamina threshold.

📊 Public output values

These are used by other systems:

- **resultingVelocity**
  → Final movement speed (normal or fast).
- **hasStamina**
  → True if the player is currently running and still has stamina available.
- **staminaPercentage**
  → Stamina converted into percentage (0–100), useful for UI bars.

🏗️ Constructors (How it is created)

There are two modes:

🟢 1. Without stamina system

- Only normal and fast speed are provided
- Stamina system is disabled
- Player can always run freely

Behavior:

- Default speed is normal
- Running speed can still be used manually

🔵 2. With stamina system

Requires:

- normal speed
- fast speed
- movement state function

Optional:

- max stamina
- minimum stamina threshold
- depletion rate
- recovery rate

When created:

- stamina is set to maximum
- running is allowed initially
- system automatically subscribes to physics updates

🔌 **Subscription system**

SubscribeUpdate

- Connects the system to the global physics update loop
- Ensures Update runs every fixed frame
- Prevents duplicate subscriptions

Dispose

Used to safely remove the system:

- Unsubscribes from update loop
- Clears reference
- Prevents memory leaks or unwanted updates

🎮 **Movement Methods**

🏃 **StartRunning**

This is called when the player presses run.

Logic:

- If stamina system is active AND player has stamina:
  - set running state ON
  - use fast speed
- Otherwise:

    - force normal speed
- If stamina system is disabled:
    - always use fast speed

👉 Result: determines whether player is walking or running

🛑 **StopRunning**

Used when:

- player releases run button
- stamina is exhausted
- movement stops

What it does:

- disables running
- resets speed to normal

🔄 **Update Logic (core stamina system)**

Runs every physics frame.

---

1. Early exit

- If stamina system is disabled → nothing happens

---

2. Reads configuration

- Gets max stamina
- Gets fixed delta time

---

3. Stamina depletion (while running)

Happens when:

- player is running
- stamina is available
- player is moving

Effect:

- stamina decreases over time

- value is clamped (never goes below 0 or above max)

If stamina reaches zero:

- running is disabled
- player is forced back to walking
- stamina is marked as exhausted
- UI becomes 0%

---

4. Stamina recovery

Happens when:

- player is not running OR not moving
- stamina is below maximum

Effect:

- stamina slowly increases over time
- value is clamped again

Recovery threshold:

- once stamina passes a minimum value:
  - player is allowed to run again

---

5. Running status update

- Checks if:
  - speed is currently fast
  - stamina is sufficient

→ updates whether player is considered “actively running”

---

6. UI value update

- Converts stamina into percentage:
  - current stamina / max stamina × 100

Used for:

- stamina bars
- HUD display
- gameplay logic

🎯 How to use this system

1. Create controller

Provide:

- speed functions
- movement state function
- optional stamina configuration

---

2. Call StartRunning

When player holds run input:

- StartRunning()

---

3. Call StopRunning

When player releases run:

- StopRunning()

---

4. Read output values

Other systems can use:

- resultingVelocity → movement system
- staminaPercentage → UI
- hasStamina → gameplay logic

---

5. Dispose when needed

When player or scene ends:

- Dispose()

⚙️ Key design idea

This system separates:

- **Speed logic**
- **Stamina logic**
- **Input state**

- **UI values**

So it can be reused for:

- player characters
- NPC stamina systems
- AI fatigue systems
- survival mechanics

🧩 Summary

This script is a **complete stamina-based movement controller** that:

- Switches between walking and running
- Drains stamina while sprinting
- Regenerates stamina when resting
- Prevents running when exhausted
- Automatically updates every physics frame
- Provides clean data for UI and gameplay
- Works independently of any specific player controller

Player Utils

This script is a small utility helper focused on **vector math for movement systems**, mainly used when you need to rotate input directions (like WASD or joystick movement) without relying on Transform rotation.

🧠 Overview of what the script does

It contains a single static utility method that:

- Takes a **2D direction vector**
- Rotates it by a given angle (in degrees)
- Returns the new rotated direction

This is commonly used in:

- Player movement systems
- Camera-relative movement
- Diagonal movement correction
- Steering logic

📦 Class: PlayerUtils

This is a **utility container class**, meaning:

- It is not meant to be instantiated (no objects are created from it)
- It only holds helper functions
- Everything inside is static and reusable anywhere

🔧 Method: ConvertRotation

🎯 Purpose

Rotates a 2D vector by a specific angle in degrees.

Example use:

- Input: “move forward”
- Camera rotated 90 degrees → forward becomes right

📥 Parameters

1. direction

This is the original movement direction.

- Example values:
  - (0, 1) → forward
  - (1, 0) → right
  - (-1, 0) → left
  - (0, -1) → backward

Think of it as:

“Where the player wants to move before rotation is applied.”

---

2. rotation

This is the angle in degrees used to rotate the vector.

- Positive values rotate clockwise
- Negative values rotate counter-clockwise

Example:

- 90 → turn forward into right
- 180 → reverse direction
- 45 → diagonal adjustment

🔁 Step-by-step logic inside the method

1. Convert degrees to radians

The math functions for sine and cosine require radians, not degrees.

So the angle is converted first.

---

2. Calculate cosine and sine

These are the core of rotation math:

- cosine → controls horizontal influence
- sine → controls vertical influence

They define how much of the original vector goes into each axis.

---

3. Apply 2D rotation formula

This is the standard 2D rotation rule:

- new X = x * cos − y * sin
- new Y = x * sin + y * cos

What this means in practice:

- The vector is “spun” around the origin
- Its length stays the same
- Only its direction changes

---

4. Return new vector

A new rotated direction is created and returned.

Nothing is modified in-place.

📤 Return value

Returns:

- A new 2D vector representing the rotated direction

Example:

- Input: (0, 1), rotation = 90

- Output: (1, 0)

🎮 **Usage Example (conceptual)**

Used when you want movement to follow camera direction:

- Player presses “forward”
- Input gives (0, 1)
- Camera is rotated 45°
- ConvertRotation adjusts movement so forward matches camera orientation

🧩 **Why this method is useful**

Without this:

- Movement feels disconnected from camera
- You’d need complex Transform logic everywhere

With this:

- Movement stays clean and reusable
- Works for any system (keyboard, joystick, AI)

Base Player 3D

This script is a **full modular 3D player controller system**. Instead of doing everything in one simple movement script, it splits responsibilities into systems (movement, jumping, rotation, stamina, collision detection, input, and state control). The result is a flexible controller where each part can be replaced or adjusted independently.

Below is a breakdown of **how it works, what each variable means, and how each method is used**.

🧠 **CORE IDEA**

The controller works like a pipeline:

Input → Sensors → State → Controllers → Physics/Transform output

Every frame it:

- Reads input
- Checks environment using sensors
- Updates player state (grounded, crouching, running, etc.)
- Calculates movement speed

- Applies movement, rotation, jumping, and stamina effects

🎮 ENUMS (PLAYER MODES)

PlayerControlMode

Defines how input is interpreted:

- Automatic → system fully controls behavior (normal gameplay)
- Manual → player control is restricted or simplified (e.g., cutscenes or special states)

PlayerAbility

Defines what the player is allowed to do:

- None → no abilities
- CanCrouch → crouching enabled
- CanJump → jumping enabled

This acts like a permission system.

PlayerMovement

Defines movement capability:

- justWalk → only walking allowed
- canRun → running allowed

⚙️ INPUT SETTINGS

These define which input actions drive the player:

- Move Action → movement direction input (like WASD or joystick)
- Run Action → run button
- Jump Action → jump button
- Crouch Action → crouch button

These are not raw inputs; they are *input references* that can be swapped in Unity.

🧍 REFERENCES (CORE OBJECTS)

- Player Transform → position/rotation of the character in the world
- Player Rigidbody → physics body used for movement and jumping

- Standing Collider → collider when upright
- Crouching Collider → collider when crouched

Only one collider is active at a time depending on posture.

🏃 MOVEMENT SETTINGS

These define movement behavior:

- Walk Speed → normal movement speed
- Run Speed → faster movement speed
- Crouch Walk Speed → slow movement while crouched
- Crouch Run Speed → faster crouch movement (rare mechanic)
- Rotation Smooth Speed → how fast the character turns
- Jump Force → strength of jump impulse
- Max Jump Count → allows double jump, triple jump, etc.
- Coyote Time Duration → small grace time after leaving ground where jump is still allowed
- Global Input Rotation Offset → rotates input direction (useful for camera-relative movement systems)

□ STAMINA SYSTEM

Controls endurance and running:

- Max Stamina → full stamina capacity
- Min Stamina For Run → threshold required to start running
- Stamina Depletion Rate → how fast stamina decreases while running
- Stamina Recovery Rate → how fast stamina regenerates

This system connects directly to running behavior.

🧠 PLAYER STATE SETTINGS

These define global gameplay rules:

- Control Mode → automatic or manual control
- Current Ability → active ability state (jump/crouch/etc.)
- Movement Mode → whether running is allowed

📦 COLLISION LAYERS

Used for filtering physics detection:

- Ground Layers → what counts as ground
- Obstacle Layers → walls and blocking objects

- Ceiling Layers → used to prevent standing up under low ceilings

---

📦 SENSOR SETTINGS

Sensors are box-shaped detection systems:

Ground Sensor

- Detects if player is standing on something
- Uses ignored tags to avoid false detection

Obstacle Sensor

- Detects walls in front of player
- Has different shape when crouching

Ceiling Sensor

- Prevents standing up if something is above player

Each sensor uses:

- position
- size
- rotation
- layer filtering
- ignored tags

🧪 PHYSICS SETTINGS

- High Friction Material → sticky ground (slower sliding)
- Low Friction Material → slippery surfaces
- Max Slope Angle → limits steep terrain walking

🐞 DEBUG SETTINGS

Enable visual debugging for:

- ground detection box
- obstacle detection box
- ceiling detection box
- slope angle visualization

🔁 RUNTIME FIELDS (INTERNAL SYSTEM STATE)

These are not editable in Inspector; they update during gameplay.

Sensors

- groundSensor → checks ground contact
- obstacleSensor → checks walls
- ceilingSensor → checks overhead blocking

Controllers

- jumpController → handles jumping logic
- moveController → handles horizontal movement
- rotationController → handles turning toward movement direction
- staminaController → handles running + stamina system

Input Event Systems

These connect Unity Input System to gameplay logic:

- jumpInputEvent
- moveInputEvent
- crouchInputEvent
- runInputEvent

They trigger actions like jump, movement, crouch toggle, and running.

Movement State Cache

- currentMoveInput → last movement input direction
- currentSpeed → final movement speed applied
- isGrounded → whether player is on ground
- isCrouching → crouch state
- isRunning → running state
- isPlayable → whether player input is enabled
- canRun → whether running is allowed

Sensor Data Cache

- cachedPlayerPosition → current position snapshot
- cachedPlayerRotation → current rotation snapshot

Used to avoid recalculating transform multiple times per frame.

Obstacle / Ceiling Position Data

- currentObstacleCenter → where obstacle check is performed
- currentObstacleSize → size of obstacle detection box
- currentCeilingCenter → ceiling detection position

Crouch Timing Control

- crouchBlock → prevents spam toggling crouch
- allowCrouchThisFrame → allows crouch only immediately after landing

---

📊 PUBLIC PROPERTIES (READ ACCESS)

These expose internal state safely:

- IsGrounded → true if touching ground
- IsCrouching → crouch state
- IsMoving → checks input + obstacle + playability
- IsRunning → running + stamina + allowed state
- StaminaPercent → stamina UI value (0–100)
- IsPlayable → whether player can be controlled

🔧 CONFIGURATION PROPERTIES

These allow external scripts or inspector tools to modify:

- Input actions (move/run/jump/crouch)
- Transform and Rigidbody references
- Colliders
- Movement speeds
- Jump settings
- Stamina settings
- Layer masks
- Physics materials
- Slope angle

This makes the controller highly modular.

🎬 UNITY LIFECYCLE METHODS

Awake

- Validates all required references are assigned
- Warns if something is missing

Start

- Initializes sensors, controllers, and input systems

Update (main loop)

Runs every frame and executes pipeline:

1. Cache transform data
2. Update sensor positions
3. Update grounded state
4. Handle abilities + crouch logic
5. Update speed + colliders
6. Update crouch timer

OnDestroy

- Cleans up all systems to prevent memory leaks or lingering subscriptions

📦 INITIALIZATION SYSTEM

ValidateSerializedReferences

Checks if important fields exist (Rigidbody, actions, colliders, etc.)

InitializeComponents

Sets default state and starts systems:

- sets movement state
- enables playable mode
- prepares sensors
- prepares controllers
- binds input

🧠 SENSOR SETUP

Creates:

- ground sensor → detects floor
- obstacle sensor → detects walls

- ceiling sensor → prevents standing up
- slope sensor → checks walkable angles

Each sensor uses dynamic lambdas so values update in real time instead of being copied once.

🎮 CONTROLLER SETUP

Connects gameplay systems:

- Jump Controller → jump logic (coyote time, multi-jump)
- Move Controller → physics movement
- Rotation Controller → smooth turning toward movement direction
- Stamina Controller → running + stamina drain/recovery

🎮 INPUT BINDING SYSTEM

Each input action has behaviors:

Jump

- triggers jump if ability allows
- activates crouch block timer

Move

- converts input into world direction (with rotation offset)
- applies movement
- resets when released

Crouch

- toggles crouch state
- prevents crouch spam
- blocks standing if ceiling detected

Run

- starts stamina drain + fast movement
- stops running when released

🔄 CORE UPDATE LOGIC

CacheTransformData

Stores position/rotation for reuse.

UpdateSensorPositions

Updates:

- obstacle detection box position/size
- ceiling detection position

Adapts dynamically when crouching.

UpdateGroundedState

Determines if player is grounded using:

- ground sensor
- ability state
- slope validation

Also detects landing transitions.

UpdateAbilityAndCrouchLogic

Handles:

- switching crouch off in manual mode
- forcing grounded state if ability is None
- tracking last valid ability
- updating run permission

UpdateSpeedAndColliders

Final movement decision:

- if crouching → use crouch speed
- otherwise → use stamina controller speed
- switches between standing/crouching collider

🌐 PUBLIC API METHODS

SetControlMode

Changes control behavior (automatic/manual)

SetPlayerAbility

Changes allowed actions (jump/crouch/etc.)

SetRunEnabled

Enables or disables running system

TogglePlayable

Enables/disables full player control

SavePlayerData

Serializes:

- position
- rotation
- state
- mode
  into a JSON string

LoadPlayerData

Restores saved state:

- control mode
- ability
- movement mode
- position
- rotation
- crouch/ground state

🧹 CLEANUP SYSTEM

DisposeAll

Removes everything safely:

- sensors
- controllers
- input events

Prevents memory leaks and dangling references.

🧩 EDITOR (UNITY ONLY)

Custom inspector groups all settings into clean sections:

- Input
- References
- Movement
- Stamina
- Collision
- Sensors
- Physics
- Debug

This improves usability in Unity Inspector.

🎯 FINAL SUMMARY

This script is essentially a:

modular character engine made of independent subsystems

Instead of a single movement script, it uses:

- sensors for environment awareness
- controllers for gameplay logic
- input events for responsiveness
- state machines for abilities and movement modes
- stamina system for realism
- save/load system for persistence

It behaves like a **mini gameplay framework for a 3D character**, not just a movement controller.

First Person Player Controller 3D
This script is a **high-level modular 3D first-person player controller for Unity**, built around a system of **sensors, input events, and separate controllers for movement, jumping, and**

stamina. I'll explain it in a structured way: what each part does, what the variables represent, and how everything works together during gameplay.

🧠 1. General Purpose of the Script

This controller handles:

- Player movement (walking, running, crouching)
- Jumping (including double jump + coyote time)
- Stamina system for running
- Collision detection using custom box sensors
- Camera movement (standing vs crouching)
- Input handling via Unity Input System
- Ability/state system (what the player is allowed to do)
- Save/load of player state

Instead of being a single monolithic script, it is **split into systems** (movement, jump, stamina, sensors), which makes it modular and extensible.

🧩 2. Enums (Core Gameplay States)

🎮 PlayerControlMode

Defines how input behaves:

- Automatic → crouch and actions are handled automatically
- Manual → player has stricter control (some actions restricted)

🧍 PlayerAbility

Defines what the player can currently do:

- None → disabled state
- CanCrouch → player is crouching
- CanJump → player can jump

This is used as a **state controller for abilities**, not just a flag.

🏃 PlayerMovement

Defines movement capacity:

- justWalk → no running allowed

- canRun → running enabled

🎛️ 3. Serialized Fields (Inspector Settings)

These are editable in Unity Inspector.

🎮 Input Settings

These store Unity Input System actions:

- moveAction → movement input (WASD / stick)
- runAction → sprint input
- jumpAction → jump input
- crouchAction → crouch input

They define **what controls the player**.

🧍 References

Core object references:

- playerTransform → player position/rotation in world
- playerRigidbody → physics body
- standingCollider → collider when standing
- crouchingCollider → collider when crouched

These are essential for physics and collision.

📷 Camera Settings

Controls camera positioning:

- cameraPivot → camera root transform
- standingCameraLocalPos → normal view position
- crouchingCameraLocalPos → lowered view position
- cameraLerpSpeed → smoothing speed between positions

Used for smooth camera transitions when crouching.

🏃 Movement Settings

Defines movement behavior:

- walkSpeed → normal speed
- runSpeed → sprint speed
- crouchWalkSpeed → crouch walking speed
- crouchRunSpeed → crouch sprint speed
- jumpForce → jump strength
- maxJumpCount → double jump limit
- coyoteTimeDuration → jump forgiveness after leaving ground

🔋 Stamina Settings

Controls running energy system:

- maxStamina → max energy
- minStaminaForRun → minimum required to sprint
- staminaDepletionRate → drain per second while running
- staminaRecoveryRate → recovery when not running

⚙️ Player State Settings

Defines current gameplay mode:

- controlMode → automatic/manual control
- currentAbility → current ability state
- movementMode → walking or running enabled

🧱 Collision Layers

Defines what objects count as:

- groundLayers → what is ground
- obstacleLayers → walls/blocks
- ceilingLayers → overhead blockers

📦 Sensors (VERY IMPORTANT SYSTEM)

The script uses **box-based collision sensors instead of Unity triggers**.

Ground Sensor

Detects if player is standing on ground.

Obstacle Sensor

Detects walls in front of player.

Ceiling Sensor

Prevents standing up when something is above the player.

Each sensor uses:

- position function
- size function
- rotation
- layer mask
- ignored tags
- debug settings

So sensors are **procedural, not static**.

🎯 Physics Materials

- highFrictionMaterial → slippery vs stable surfaces
- lowFrictionMaterial → low grip surfaces

Used for slope behavior.

🐞 Debug Settings

Enable visual gizmos for:

- ground detection
- obstacle detection
- ceiling detection
- slope angle validation

🧠 4. Runtime Fields (Internal State)

These are calculated during gameplay:

Sensors

- groundSensor → detects floor
- obstacleSensor → detects walls
- ceilingSensor → detects overhead block
- slopeAngleSensors → checks if slope is walkable

Controllers (Core Logic Modules)

Instead of putting logic in one script:

- jumpController → handles jumping physics
- moveController → handles movement + collisions
- staminaController → handles speed + stamina system

This is a **component-based logic architecture inside one MonoBehaviour**.

Input Events

- jumpInputEvent → jump input handler
- moveInputEvent → movement handler
- crouchInputEvent → crouch logic
- runInputEvent → sprint logic

These wrap Unity Input System and attach behavior to events.

Cached Data (Performance Optimization)

- cachedPlayerPosition → avoids repeated transform calls
- cachedPlayerRotation → same for rotation

Movement State Variables

- currentMoveInput → current WASD direction
- currentSpeed → final movement speed applied
- isGrounded → grounded state
- isCrouching → crouch state
- isRunning → sprint state
- isPlayable → whether input is allowed
- canRun → whether sprint is allowed

Crouch Control Lock

- crouchBlock → prevents spamming crouch (cooldown timer)
- allowCrouchThisFrame → special landing logic

5. Public Properties (Read Access)

These expose internal state safely:

- IsGrounded → grounded status
- IsCrouching → crouch status
- IsMoving → movement + input check
- IsRunning → sprint + stamina check
- StaminaPercent → stamina percentage
- IsPlayable → input enabled flag

These are **read-only gameplay queries**.

⚙️ 6. Initialization Flow

Awake()

- validates references (checks missing assignments)

Start()

- initializes everything:
  - sensors
  - controllers
  - input bindings

📦 7. Sensor Setup (Core System)

Three main box sensors:

Ground Sensor

Detects floor contact and ignores certain tags.

Obstacle Sensor

Detects walls in movement direction.

Ceiling Sensor

Prevents standing up under low ceilings.

🧠 8. Controller Setup

Jump Controller

Handles:

- jump force

- grounded check
- double jump
- coyote time

Move Controller

Handles:

- horizontal movement
- obstacle blocking

Stamina Controller

Handles:

- running speed
- stamina drain
- stamina recovery
- final movement velocity

🎮 9. Input System Setup

Jump Input

On press:

- triggers jump
- activates cooldown block

Movement Input

On hold:

- reads movement vector
- converts based on camera rotation
- sends to movement controller

On release:

- stops movement

Crouch Input

- toggles crouch state (if allowed)
- checks ceiling before standing
- respects control mode

Run Input

- starts/stops sprint
- activates stamina system

🔄 10. Main Update Loop (Every Frame)

Order of operations:

1. Cache transform data
2. Update sensor positions
3. Check grounded state
4. Update ability logic
5. Update speed + colliders
6. Move camera smoothly
7. Update crouch cooldown timer

This ensures **everything stays synchronized per frame**.

🧭 11. Movement & Physics Logic

Sensor Positioning

- obstacle sensor moves based on input direction
- ceiling sensor follows player
- crouch changes sensor size

Ground Check Logic

Player is grounded if:

- ground sensor detects collision OR ability disallows jumping
- AND slope is valid OR player is crouching

Also handles:

- landing detection for crouch timing

Ability System Logic

- manual mode forces crouch off
- none ability disables movement
- updates last valid non-crouch state

Speed System

Speed is decided by:

- crouching or not
- running state
- stamina controller output

Also:

- toggles colliders depending on stance

Camera System

- smooth interpolation between standing and crouching positions
- prevents abrupt camera jumps

💾 **12. Save / Load System**

SavePlayerData

Stores:

- control mode
- ability
- movement mode
- position
- rotation
- grounded state
- crouch state

Converted into JSON.

LoadPlayerData

Restores:

- player state
- transforms position

- ability system state

🧹 13. Cleanup (OnDestroy)

Properly disposes:

- all sensors
- all controllers
- input event subscriptions

Prevents memory leaks and dangling references.

👨‍💻 14. Editor Custom Inspector

This part only runs in Unity Editor.

It organizes Inspector into clean groups:

- Input Settings
- References
- Camera
- Movement
- Stamina
- States
- Collision Layers
- Sensors
- Physics
- Debug

This improves usability dramatically.

🧠 Final Summary (Big Picture)

This script works like a **mini game engine inside one player controller**, composed of:

- 🎮 Input layer (Unity Input System)
- 🧠 State system (abilities + movement modes)
- 🔭 Sensor system (custom collision detection)
- ⚙️ Controller system (movement/jump/stamina)
- 📷 Camera system (smooth transitions)
- 💾 Save/load system
- 🧹 lifecycle cleanup

Instead of relying on Unity's default character controller, it builds a **fully custom modular physics-based player controller** with strong separation of concerns.

First Person Camera Controller

This script is a **first-person camera controller**. Its job is simple in concept:
it reads input (mouse or stick movement) and converts it into two rotations:

- Horizontal rotation (turning left/right) → applied to the **player body**
- Vertical rotation (looking up/down) → applied to the **camera pivot**

It also controls sensitivity, inversion, and vertical angle limits so the camera doesn't rotate unnaturally.

🔧 Main idea of how it works

Every frame:

1. It reads look input from an input handler.
2. It multiplies that input by sensitivity settings.
3. It splits the result into:
    - yaw (left/right)
    - pitch (up/down)
4. It rotates:
    - the player for yaw
    - the camera pivot for pitch (clamped)

📦 References (Serialized Fields)

These are the external links the script needs to function:

🎮 Look Input Handler

- Provides raw camera input (mouse or right stick direction).
- Without it, the camera has no control input.

Usage:
It supplies a 2D direction vector (like X = left/right, Y = up/down).

🧍 Player Transform

- Represents the entire player body.
- Only rotates on the vertical axis (turning left/right).

Usage:

- When you move the mouse horizontally, the whole character turns.

🎥 Camera Pivot

- A child transform of the player.
- Controls vertical rotation only.

Usage:

- Makes the camera look up and down without rotating the whole body.

🎛️ Camera Settings

🎯 Camera Sensitivity

- Global multiplier for how fast the camera moves.

Usage:

- Higher value = faster camera movement
- Lower value = smoother/slower movement

⚖️ Angle Sensitivity (X and Y)

- Separates control over:
  - X = horizontal speed (yaw)
  - Y = vertical speed (pitch)

Usage:

- Lets you fine-tune:
  - fast turning left/right
  - slower looking up/down (or vice versa)

⬇️ Min Pitch / ⬆️ Max Pitch

- Limits how far the camera can look up or down.

Usage:

- Prevents full rotation or neck-breaking angles
- Example:
  - min = -80 → can look almost straight down
  - max = 80 → can look almost straight up

🔁 Invert Y

- Flips vertical input direction.

Usage:

- If true:
  - moving mouse up makes camera look up (or inverted depending on system preference)

🧠 Runtime Field

currentPitch

- Stores the current vertical rotation of the camera.

Usage:

- Keeps track of how far the camera has already tilted
- Updated every frame
- Ensures smooth continuous rotation instead of resetting

🧩 Properties (Public Access)

These allow other scripts to read or modify settings safely.

Examples:

Look Input Handler Property

- Lets you assign or replace the input source dynamically.

Player Transform Property

- Allows external systems to change which object rotates horizontally.

Camera Pivot Property

- Lets other scripts control or replace the camera target.

Sensitivity / Angle Settings

- Allows runtime adjustment of camera feel.

Pitch Limits

- Lets you change how far the player can look up/down during gameplay.

Invert Y Property

- Enables toggling inversion in settings menus.

⚙ Unity Lifecycle

Awake (initial validation)

Runs once when the object is created.

It checks:

- If references are missing → warns the developer
- If minPitch > maxPitch → throws an error (invalid configuration)

Purpose:
Prevents broken camera setup before gameplay starts.

Update (every frame)

Calls the main function:

➞ HandleMouseLook

Purpose:
Keeps camera responsive in real time.

🎮 Core Logic: HandleMouseLook

This is the heart of the system.

Step 1: Safety check

If any required reference is missing → stop execution.

Step 2: Read input

- Gets 2D input from the input system
- Multiplies it by sensitivity

Result:

- X = horizontal movement
- Y = vertical movement

Step 3: Apply sensitivity scaling

- Horizontal uses angleSensitivity.x
- Vertical uses angleSensitivity.y

Step 4: Invert Y logic

- If invertY is enabled, pitch direction is flipped

Step 5: Rotate player (yaw)

- Player rotates around the Y axis

Effect:

- Turning left/right rotates the entire character

Step 6: Update camera pitch

- Adds vertical input to currentPitch
- Clamps it between minPitch and maxPitch

Effect:

- Prevents over-rotation

Step 7: Apply rotation to camera pivot

- Sets local rotation of camera pivot on X axis only

Effect:

- Camera looks up/down smoothly

🛠 Utility Methods

SetSensitivity(value)

- Safely changes sensitivity
- Clamped between 0.05 and 1

Usage:

- Used in settings menus or gameplay adjustments

SetInvertY(enabled)

- Enables or disables inverted vertical control

Usage:

- Often used for accessibility or player preference

🎯 Summary of behavior

- Mouse movement → input vector
- Horizontal input → rotates player body
- Vertical input → rotates camera pivot
- Pitch is clamped to avoid over-rotation
- Sensitivity controls speed
- Optional inversion flips vertical axis

Side View Player Controller 3D

This script is a **complete modular 3D side-view player controller system for Unity**, designed to handle movement, jumping, crouching, running, stamina, collision detection, and state management in a very structured way.

Below is a breakdown of how it works, focusing on **purpose, variables, methods, and usage (without C# syntax)**.

🧠 Overall Idea

The controller works like a **central brain** that:

- Reads input (movement, jump, crouch, run)

- Checks environment (ground, obstacles, ceiling, slope)
- Controls physics movement (via Rigidbody)
- Manages player state (crouching, running, stamina, abilities)
- Uses modular systems (sensors + controllers)

Everything is split into:

- **Configuration (serialized fields)**
- **Runtime state (internal variables)**
- **Systems (sensors + controllers)**
- **Input events**
- **Update loop logic**

📦 1. Enums (Gameplay Rules)

These define fixed gameplay states.

Player Control Mode

- Automatic → player behaves normally (default gameplay rules)
- Manual → external systems override control (useful for cutscenes or AI control)

Player Ability

- None → no actions allowed
- CanCrouch → crouch enabled
- CanJump → jump enabled

Movement Mode

- justWalk → only walking allowed
- canRun → running enabled

⚙️ 2. Serialized Fields (Inspector Settings)

These are **editable in Unity Inspector** and define how the controller behaves.

🎮 Input Settings

- Move input → directional movement (keyboard/joystick)
- Run input → sprint button
- Jump input → jump trigger
- Crouch input → crouch toggle

👉 These define how player input is received.

🧍 References

- Player transform → main object position and rotation
- Rigidbody → physics movement engine
- Standing collider → normal hitbox
- Crouching collider → smaller hitbox when crouched

👉 These are required components of the player.

🏃 Movement Settings

Defines movement behavior:

- Walk speed → normal movement speed
- Run speed → faster movement speed
- Crouch walk/run speed → reduced movement while crouched
- Rotation smooth speed → how smoothly the player turns
- Jump force → strength of jump
- Max jump count → allows double jump or more
- Coyote time → small delay allowing jump after falling
- Global input rotation offset → rotates input direction globally

---

🧠 Stamina System

- Max stamina → total energy
- Min stamina for run → threshold required to sprint
- Depletion rate → how fast stamina drains while running
- Recovery rate → how fast stamina regenerates

👉 Controls sprint endurance system.

🧍 Player State Settings

- Control mode → automatic/manual behavior
- Current ability → what the player can do
- Movement mode → walking or running allowed

🧱 Collision Layers

Defines what the sensors detect:

- Ground layers → surfaces considered walkable
- Obstacle layers → walls or blocking objects

- Ceiling layers → prevents standing up in crouch

📡 Sensor Settings

Ground sensor

- Detects if player is touching ground
- Uses box shape for detection

Obstacle sensor

- Detects walls in front of player
- Has different size when crouching

Ceiling sensor

- Detects overhead blocking objects

Each sensor also has:

- Size
- Position offset
- Ignored tags (like “Ignore Collision”)

🧪 Physics Materials

- High friction → sticky surfaces
- Low friction → slippery surfaces

🐞 Debug Options

- Show ground detection
- Show obstacle detection
- Show ceiling detection
- Show slope angle debug

👉 Used for visualization in editor.

🔄 3. Runtime Fields (Internal State)

These are **computed during gameplay**.

Sensors

- Ground sensor → detects floor contact
- Obstacle sensor → detects walls
- Ceiling sensor → detects roof collision
- Slope sensor → checks if surface is walkable

Controllers

- Jump controller → handles jump logic
- Move controller → handles movement
- Rotation controller → rotates player toward movement
- Stamina controller → manages running energy

Input Events

- Jump event → reacts to jump button
- Move event → reacts to movement input
- Crouch event → toggles crouch
- Run event → enables sprinting

Cached Data

- Player position → stored each frame
- Player rotation → stored each frame
- Movement input → current direction input

Movement State

- Current speed → final speed applied
- Is grounded → on ground or not
- Is crouching → crouch state
- Is running → sprint state
- Can run → allowed to run
- Is playable → whether input is allowed

Crouch Timer

- Prevents instant spam toggling crouch
- Adds small delay between crouch actions

🧭 **4. Public Properties (External Access)**

These allow other scripts to:

- Check state (grounded, crouching, moving, running)
- Read stamina percentage
- Enable/disable player control
- Modify speeds, stamina, and settings

👉 Example usage:

- UI stamina bar reads stamina percent
- Animation system checks IsRunning
- Game manager disables IsPlayable during cutscenes

🚀 5. Unity Lifecycle Methods

Awake (setup validation)

Runs when object is created:

- Checks if required references exist
- Warns if something is missing

👉 Prevents broken setup.

Start (initialization)

Runs before first frame:

- Sets default states
- Initializes sensors
- Initializes controllers
- Binds input actions

👉 Prepares player system.

Update (main loop)

Runs every frame and executes:

1. Cache player transform data
2. Update sensor positions
3. Check if grounded
4. Update ability/crouch logic
5. Update movement speed and colliders
6. Update crouch timer

👉 This is the **core brain loop of the controller**.

OnDestroy (cleanup)

When object is destroyed:

- Disposes sensors
- Disposes controllers
- Unbinds input events

👉 Prevents memory leaks and leftover input listeners.

🧩 6. Setup Methods

Validate References

Checks all required components exist.

Initialize Components

Sets initial values:

- Movement state
- Ability state
- Sensor setup
- Controller setup
- Input setup

Setup Sensors

Creates detection systems:

- Ground detection box
- Obstacle detection box
- Ceiling detection box
- Slope validation system

👉 These control environmental awareness.

Setup Controllers

Creates gameplay systems:

- Jump behavior
- Movement behavior
- Rotation behavior
- Stamina system

👉 These control player physics and logic.

Setup Input Bindings

Maps input actions:

Jump

- On press → jump if allowed

Move

- On hold → update direction
- On release → stop movement

Crouch

- Toggles crouch state
- Prevents crouch if ceiling blocks

Run

- On press → start sprint + drain stamina
- On release → stop sprint

🔄 7. Update Logic Methods

Cache Transform Data

Stores:

- Position
- Rotation

Used by sensors.

Update Sensor Positions

Adjusts:

- Obstacle sensor (changes when crouching)
- Ceiling sensor position

Update Ground State

Determines:

- If player is grounded
- If slope is valid
- If crouch is allowed after landing

Update Ability Logic

Handles:

- Switching between crouch and normal states
- Preventing invalid states (like crouch in manual mode)
- Updating ability history

Update Speed and Colliders

- Calculates final movement speed:
  - crouching + running → crouch run speed
  - crouching + walking → crouch walk speed
  - normal → stamina-based speed
- Enables correct collider:
  - standing collider OR crouching collider

🎮 8. Public API (External Control)

These methods let other systems control the player.

Set Control Mode

- Switch between automatic and manual control
- Prevents crouch in manual mode

Set Player Ability

- Enables/disables jump or crouch
- Prevents invalid combinations

Set Run Enabled

- Enables or disables sprint system

Toggle Playable

- Enables or disables all input

Save Player Data

Creates a saved snapshot:

- position
- rotation
- abilities
- movement mode
- crouch state
- grounded state

👉 Used for save systems.

Load Player Data

Restores:

- player position
- rotation
- ability state
- crouch state
- grounded state

👉 Used for loading game state.

🧹 9. Cleanup System

Disposes:

- all sensors
- all controllers
- all input bindings

👉 Important for preventing:

- memory leaks
- stuck input events
- leftover physics checks

🧪 10. Editor Script (Custom Inspector)

Improves Unity Inspector UI by:

- grouping settings logically
- organizing sections:
  - input
  - movement
  - stamina
  - sensors
  - debug

👉 This does NOT affect gameplay, only editor usability.

🧭 Summary (How It Works in Practice)

At runtime:

1. Player presses input
2. Input system triggers events
3. Sensors detect environment
4. Controllers calculate movement
5. Update loop applies physics
6. Stamina and state adjust behavior
7. Colliders and speed update dynamically

Third Person Player Controller 3D

This script is a **modular 3D player controller system for Unity**, built like a full “movement framework” instead of a simple controller. It handles input, movement, physics, stamina, crouching, jumping, camera behavior, and collision detection through separate sensor and controller modules.

Below is a structured explanation of how it works, focusing on **variables, methods, and usage (without C# syntax).**

🧠 1. General Architecture

The system is split into 4 main layers:

- **Input Layer** → reads player actions (move, jump, run, crouch)
- **Sensor Layer** → detects environment (ground, obstacles, ceiling, slope)
- **Controller Layer** → applies logic (movement, jump, rotation, stamina)
- **State Layer** → stores current player status (is grounded, crouching, running)

Everything is modular, meaning each system can be swapped or extended independently.

🧩 2. Enums (Player Rules & Modes)

🎮 Control Mode

Defines how input is interpreted:

- Automatic → system manages crouch/abilities automatically
- Manual → player has stricter control, less automatic behavior

---

🦶 Player Ability

Defines what the player is currently allowed to do:

- None → no actions
- CanCrouch → crouch enabled
- CanJump → jumping enabled

This acts like a **permission system**.

🚶 Movement Mode

Defines movement capability:

- justWalk → only walking allowed
- canRun → running allowed

⚙️ 3. Input Variables (Player Controls)

These store Unity input actions:

- Move input → directional movement (WASD / stick)
- Run input → sprint trigger
- Jump input → jump action
- Crouch input → crouch toggle

👉 These do not move the player directly
They only **send signals to the system**

🧍 4. Core References (Unity Components)

These are required scene components:

- Player transform → position/rotation of player
- Rigidbody → physics movement engine
- Standing collider → normal height hitbox
- Crouching collider → reduced hitbox

👉 Only one collider is active at a time depending on posture.

🎥 5. Camera System

- Camera pivot → local camera movement (follow player)
- World camera pivot → global alignment reference

Positions:

- Standing camera position → normal height view
- Crouching camera position → lowered view

Camera smoothly interpolates between them using:

- camera smoothing speed

👉 This avoids sudden camera jumps when crouching.

🏃 6. Movement Settings

Defines how movement behaves:

- Walk speed → normal movement speed
- Run speed → sprint speed
- Crouch walk speed → slow crouch movement
- Crouch run speed → faster crouch movement (rare case)

Other movement rules:

- rotation smooth speed → how fast player turns toward direction
- jump force → vertical force applied when jumping
- max jump count → allows double jump or more
- coyote time → short grace time after leaving ground

7. Stamina System

Controls sprint limitation:

- max stamina → full energy
- minimum stamina for running → threshold to allow sprint

Drain/recovery:

- stamina depletion rate → how fast sprint drains energy
- stamina recovery rate → how fast stamina regenerates

👉 Running depends directly on stamina availability.

🧍 8. Player State Variables

These define real-time status:

- control mode → current input behavior type
- current ability → active movement capability
- movement mode → walk/run permission state

Runtime flags:

- is grounded → touching ground or not
- is crouching → crouch state
- is running → sprint state
- is playable → player input enabled or disabled
- can run → whether sprint is allowed right now

🧪 9. Sensor System (Collision Detection)

This is one of the most important parts.

Ground Sensor

Detects:

- if player is touching ground
- if jumping is possible
- valid surfaces

Also ignores certain tags.

Obstacle Sensor

Detects:

- walls in front of player
- prevents walking through objects

Changes size when crouching.

Ceiling Sensor

Detects:

- objects above player
- prevents standing up inside low ceilings

Slope Sensor

Checks:

- if ground angle is walkable
- prevents walking on steep slopes

🎮 10. Controllers (Core Logic Systems)

Jump Controller

Handles:

- jump force
- grounded check
- coyote time
- multi-jump logic

Movement Controller

Handles:

- applying movement direction
- blocking movement when obstacle detected

Rotation Controller

Handles:

- turning player toward movement direction
- smoothing rotation over time

Stamina Controller

Handles:

- stamina drain during running
- stamina recovery when not sprinting
- calculates final movement speed

🎯 11. Input Event System

Each input is wrapped into event behavior:

Movement input

- while held → updates movement direction
- when released → stops movement

Jump input

- when pressed → triggers jump if allowed

Crouch input

- toggles crouch state
- blocked briefly after actions (prevents spam)

Run input

- pressed → starts sprint + stamina drain
- released → stops sprint + stamina recovery

🔄 12. Unity Lifecycle Methods

Awake

- checks if required references are assigned

Start

- initializes all systems (sensors, controllers, input)

Update (main loop)

Runs every frame:

1. Saves player position/rotation
2. Updates sensor positions
3. Checks ground state
4. Updates crouch and ability rules
5. Updates movement speed + colliders
6. Updates camera position
7. Updates crouch cooldown timer

👉 This is the “brain loop” of the controller

OnDestroy

- cleans everything (prevents memory leaks)

Gizmos (Editor only)

- visual debug of camera positions in editor
- shows standing vs crouching camera height

⚙️ 13. Initialization Flow

When the system starts:

1. Validate references (checks missing components)
2. Initialize variables
3. Setup sensors
4. Setup movement controllers
5. Setup input bindings

👉 This builds the entire system at runtime.

🧠 14. Runtime Update Logic (Key Behavior)

Ground detection logic:

Player is considered grounded if:

- ground sensor detects collision OR
- ability forces grounded state
  AND
- slope is valid OR crouching

Crouch logic:

- toggled by input
- blocked if ceiling detected
- restricted depending on control mode

Speed logic:

Final speed depends on:

- crouch state
- running state
- stamina controller output

Collider switching:

- standing → normal collider active
- crouching → low collider active

Camera movement:

- smoothly moves between standing/crouching positions

💾 15. Save / Load System

Save:

Stores:

- control mode
- ability state
- movement mode
- position
- rotation
- grounded state

- crouch state

Output:

- JSON string

Load:

Restores:

- all saved values
- player position and rotation
- crouch and grounded states

🧹 16. Cleanup System

When object is destroyed:

- removes sensors
- removes controllers
- unsubscribes input events

👉 Prevents memory leaks and ghost inputs.

📜 17. Public API (External Control)

These functions allow external scripts to control the player:

- Set control mode
- Set ability (jump/crouch rules)
- Enable/disable running
- Enable/disable player control
- Save player state
- Load player state

🎯 Final Summary

This script is essentially a:

🧩 Modular player simulation framework combining physics, input, sensors, stamina, and state machines.

Instead of one monolithic controller, it uses:

- sensors (environment awareness)
- controllers (logic execution)
- input events (user interaction)
- state system (behavior rules)

Third Person Camera Controller

This script is a **third-person camera controller system** for Unity. It handles **camera rotation, zoom, and collision avoidance** so the camera follows the player smoothly without clipping through walls.

Below is a structured explanation of how it works, covering **variables, methods, and usage**, without code syntax.

🎮 OVERVIEW OF THE SYSTEM

The camera is built around 3 main ideas:

- **Rotation (looking around the player)**
- **Zoom (moving camera closer/farther)**
- **Collision handling (preventing camera clipping into walls)**

It uses Unity's Input System and a custom input wrapper to react to player controls.

📦 INPUT & REFERENCE VARIABLES

🎯 Input Settings

Zoom Input Action

- Controls camera zoom in/out.
- When the player scrolls or uses input, it changes camera distance.

🔗 References

Look Input Handler

- Provides raw mouse or analog stick movement.
- This is what drives camera rotation.

Player Transform

- The main position of the player in the world.
- Camera always follows this.

Player Pivot Transform

- A stable point on the player used as the camera target position.

Main Camera Pivot Transform

- The object that actually rotates the camera.

Collision Camera Pivot Transform

- A second camera pivot used specifically for collision-safe positioning.

🎥 CAMERA SETTINGS VARIABLES

🧭 Rotation Settings

Camera Sensitivity

- Global multiplier for how fast camera rotates.

Angle Sensitivity

- Separate control for:
  - horizontal movement (yaw)
  - vertical movement (pitch)

Min / Max Pitch

- Limits how far up or down the camera can look.

Invert Y

- Reverses vertical input direction.

🔍 DISTANCE / ZOOM SETTINGS

Min Distance / Max Distance

- Defines how far the camera can zoom out or in.

Min Collision Distance

- Prevents camera from getting too close to walls.

Zoom Speed

- How fast zoom changes when input is used.

Collision Smooth Speed

- How smoothly camera reacts to obstacles.

Target Camera Distance

- Desired zoom distance before collision correction.

📦 COLLISION SETTINGS

Collision Layers

- Defines what objects block the camera (walls, environment, etc.).

Collision Offset

- Small buffer to prevent camera from touching geometry.

🧠 INTERNAL (PRIVATE) VARIABLES

🧭 Rotation State

- Current pitch angle → up/down camera angle
- Current yaw angle → left/right rotation

🎥 Camera State

- Current camera distance → actual smoothed zoom distance
- Camera active flag → enables/disables camera system

⚡ PERFORMANCE BUFFER

- Raycast hit buffer → reused array to detect collisions efficiently without memory allocation

---

🎮 INPUT EVENT

- Zoom input event → connects input system to zoom logic

🔁 UNITY LIFECYCLE METHODS

Awake (Initialization Check)

Purpose:

- Validates if all required references are assigned
- Prevents runtime errors

Checks:

- Missing input actions
- Missing transforms
- Invalid min/max values

Start (Initial Setup)

Purpose:

- Initializes camera distance
- Sets up zoom input system
- Performs first collision check

What happens:

- Camera distance is clamped to valid range
- Zoom input is linked to camera zoom behavior
- Immediate collision detection prevents starting inside walls

---

OnDestroy (Cleanup)

Purpose:

- Frees input event resources
- Prevents memory leaks

Update (Every Frame)

Runs two core systems:

1. Look Input Processing

- Handles camera rotation based on player input

2. Collision Processing

- Adjusts camera position if something blocks it

OnDrawGizmosSelected (Debug)

Purpose:

- Visual debugging in Unity editor

Shows:

- Camera pivot position
- Collision pivot position
- Line between them

🎮 CAMERA LOGIC METHODS

🧭 Process Look Input

Purpose:
Handles camera rotation.

What it does:

1. Moves camera pivot to player position
2. Reads input from look system
3. Applies sensitivity scaling
4. Updates:
    - horizontal rotation (yaw)
    - vertical rotation (pitch)
5. Clamps vertical rotation to prevent over-rotation
6. Applies final rotation to camera pivot

📌 Result:
The camera smoothly follows mouse or stick movement around the player.

🧱 Process Camera Collision

Purpose:
Prevents camera from going through walls.

Step-by-step:

1. Calculate Desired Camera Position

- Based on current rotation and zoom distance

2. Cast a Ray

- Shoots a ray from player to camera position
- Detects obstacles in the way

3. Analyze Hits

- Finds closest valid obstacle
- Ignores player's own body

4. Adjust Distance

- If obstacle found:
  - Moves camera closer to player
  - Keeps a safe offset from walls

5. Smooth Movement

- Camera moves gradually instead of snapping

6. Apply Position

- Updates camera pivot's local Z position

📌 Result:
Camera slides forward when near walls instead of clipping through them.

🌐 **PUBLIC API METHODS (USAGE CONTROL)**

These methods allow external scripts or UI to control the camera.

🎚 **Set Camera Sensitivity**

- Changes how fast camera rotates
- Automatically clamped to safe range

📌 Usage:
Used in settings menus or gameplay adjustments.

🔁 Set Invert Y Axis

- Reverses vertical camera movement

📌 Usage:
Accessibility or player preference option.

🎮 Set Camera Active

- Enables or disables entire camera system

When disabled:

- No rotation
- No collision updates
- No input processing

📌 Usage:
Cutscenes, menus, dialogue systems.

🎯 HOW IT WORKS IN PRACTICE

1. Player moves mouse → camera rotates around player
2. Player scrolls → camera zooms in/out
3. Wall appears behind player → camera moves forward automatically
4. Player walks around → camera always stays stable and smooth

---

🧠 SIMPLE SUMMARY

This system creates a:

- Smooth third-person camera
- With full rotation control
- Zoom control
- And intelligent wall collision avoidance

All while keeping movement stable and performance-friendly.